

Software Engineering II

Kommunikationsmuster

Mathis Neunzig
ABAP Developer at SAP
mathis.neunzig@gmail.com



Disclaimer

- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesung sind kein Ersatz für Literatur und können nicht als Quellen für wissenschaftliche Arbeiten genutzt werden.
- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesungen dürfen nur von den Studierenden des Kurses WI 2023 SE B zur Nachbereitung der Vorlesung genutzt werden. Eine kommerzielle Nutzung und Weitergabe ist verboten.
- Alle Präsentation sowie alle weiteren Materialien werden vor Beginn der jeweiligen Vorlesung hochgeladen.
- Zur besseren Lesbarkeit wird auf die gleichzeitige Nutzung von weiblichen & männlichen Begriffen verzichtet. Es wird das generische Maskulinum verwendet. Dabei sind weibliche und sonstige Formen des Wortes gleichwertig mitgemeint. (Beispiel: statt "Kund:innen" → „Kunden“)

REST

Was ist REST?

REST

- Representational State Transfer
- Paradigma für Softwarearchitektur
- Nutzt HTTP-Methoden (GET, POST, PUT, DELETE, ...)
- Abstraktion der Struktur und des Verhaltens des WWW

REST

Methode	Beschreibung
GET	Fordert die angegebene Ressource vom Server an
POST	Fügt eine neue (Sub-)Ressource unterhalb der angegebenen Ressource ein
PUT	Die angegebene Ressource wird angelegt bzw. geändert.
PATCH	Ein Teil der angegebenen Ressource wird geändert
DELETE	Löscht die angegebene Ressource
HEAD	Fordert Metadaten zu einer Ressource an
OPTIONS	Prüft, welche Methoden auf einer Ressource zur Verfügung stehen
CONNECT	Dient dazu, die Anfrage durch einen TCP-Tunnel zu leiten. Wird meist eingesetzt, um eine HTTPS-Verbindung über einen HTTP-Proxy herzustellen.
TRACE	Gibt die Anfrage zurück, wie sie der Zielserver erhält. Dient etwa dazu, um Änderungen der Anfrage durch Proxyserver zu ermitteln.

REST

■ Prinzipien

- Zustandslos = Jede Anfrage hat alle Informationen, nichts wird gespeichert
- Trennung von Client und Server
- Cache-fähig = Daten könnten gecached werden
- Schichtsystem = Zwischenserver können zur Skalierung genutzt werden
- Einheitliche Schnittstellen

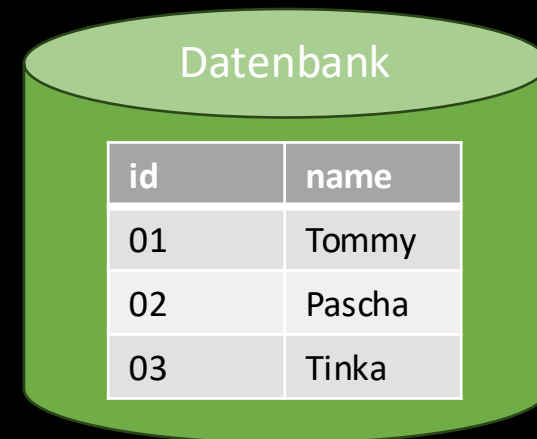
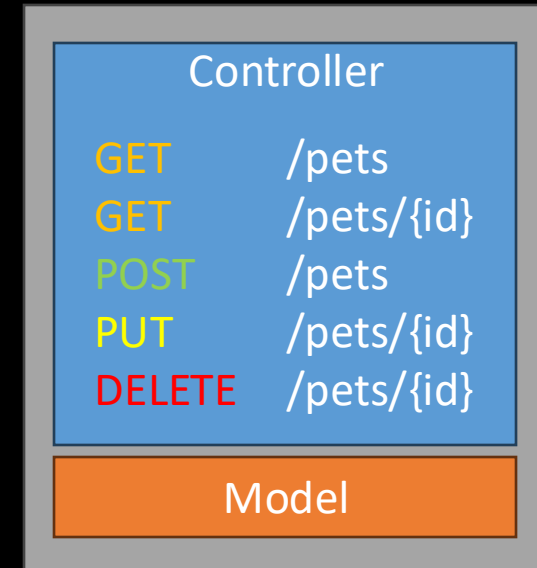
■ Praxis

- Ressourcen werden als URLs dargestellt
 - localhost/items
 - localhost/items/{id}
 - localhost/items/{id}/orders
 - ...

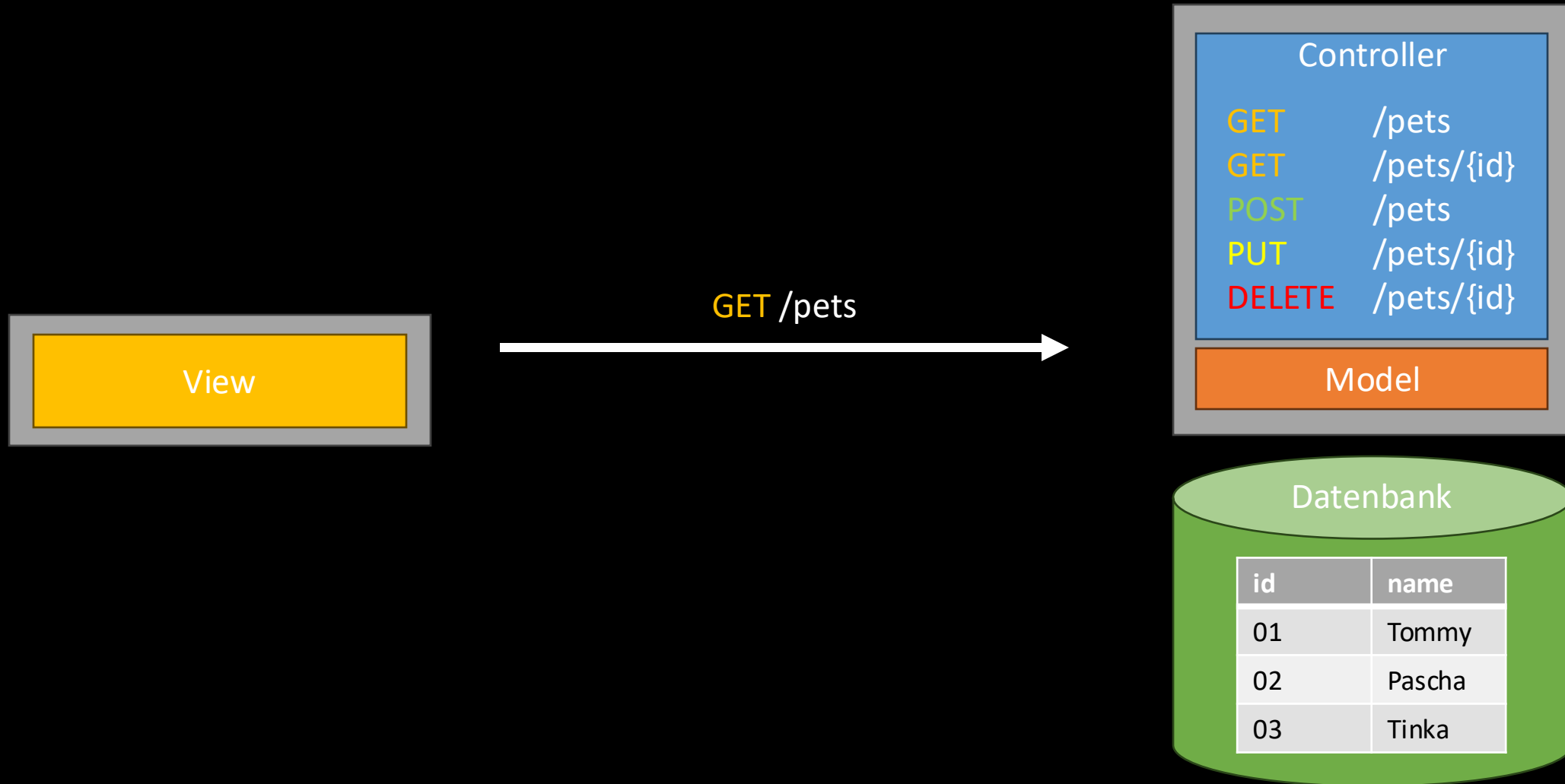
HTTP-Header

- Metadaten zur Anfrage/Antwort
- Request-Header
 - Accept: Gibt das gewünschte **Datenformat** an (z.B. application/json)
 - Authorization: Enthält **Auth-Daten** (z.B. Bearer-Token, Basic Auth)
 - Content-Type: Beschreibt das **Format** des Anfrage-Bodys (z.B. application/xml)
 - User-Agent: Kennzeichnet den **Client** (z.B. Browser, spezifisches Tool)
- Response-Header
 - Content-Type: Beschreibt das **Format** der Antwort (z.B. application/json)
 - Location: Verweist auf den **Ort einer neu angelegten Ressource** (z.B. bei POST)
 - Cache-Control: Steuert das **Caching** (z.B. no-cache, max-age=3600)
 - WWW-Authenticate: Zeigt dem Client an, welche **Auth-Methode** gefordert wird

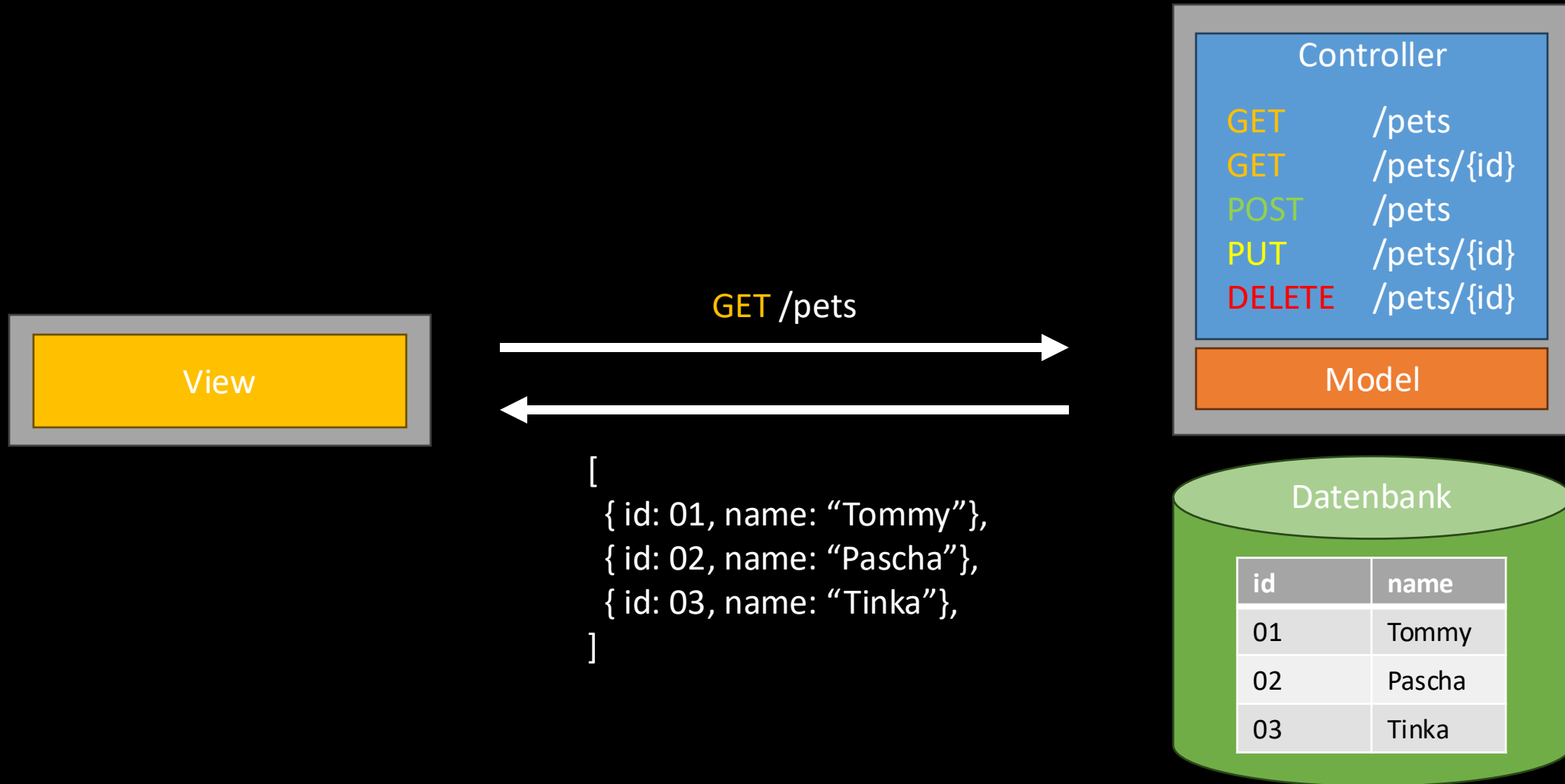
REST



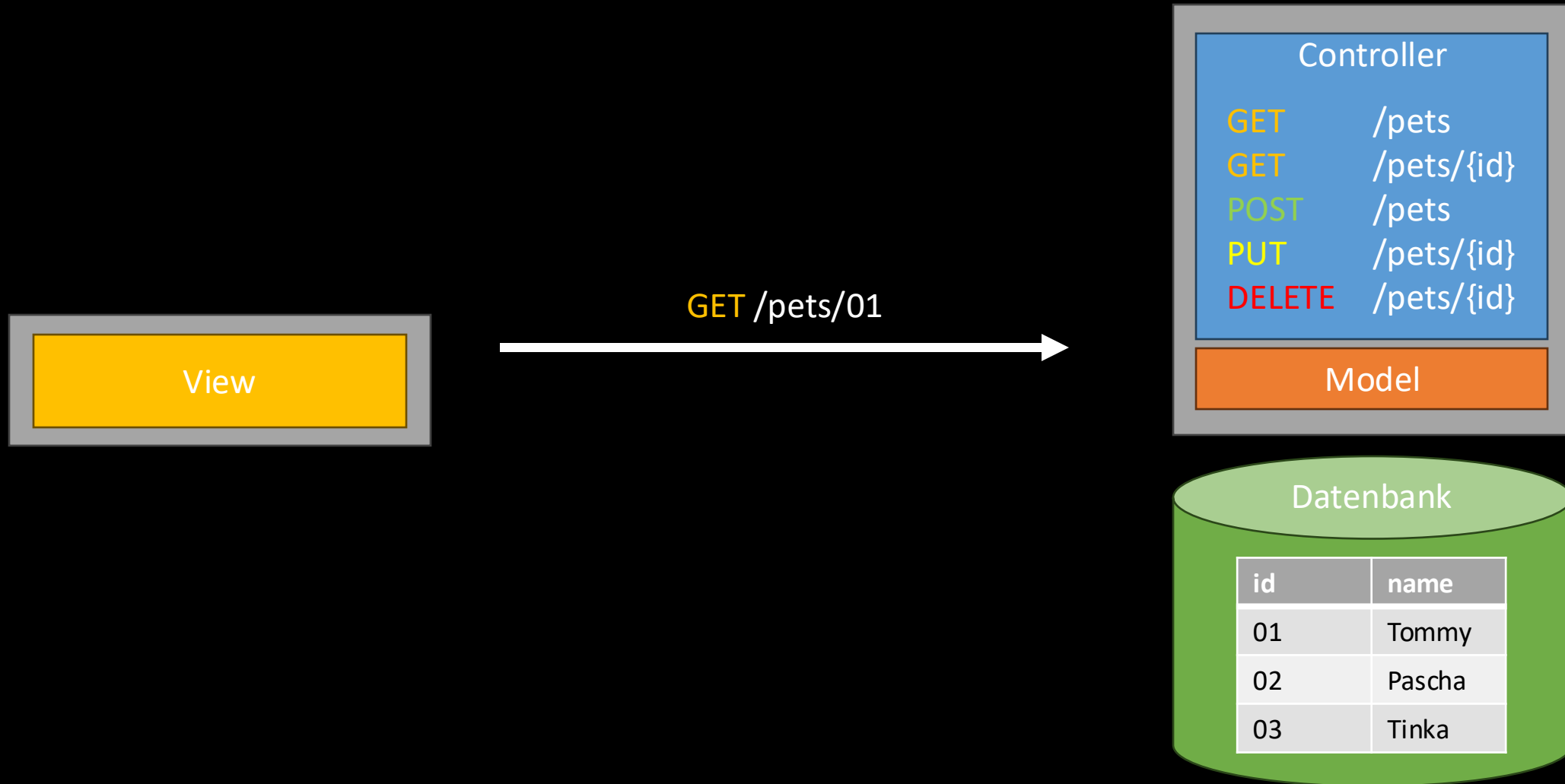
REST



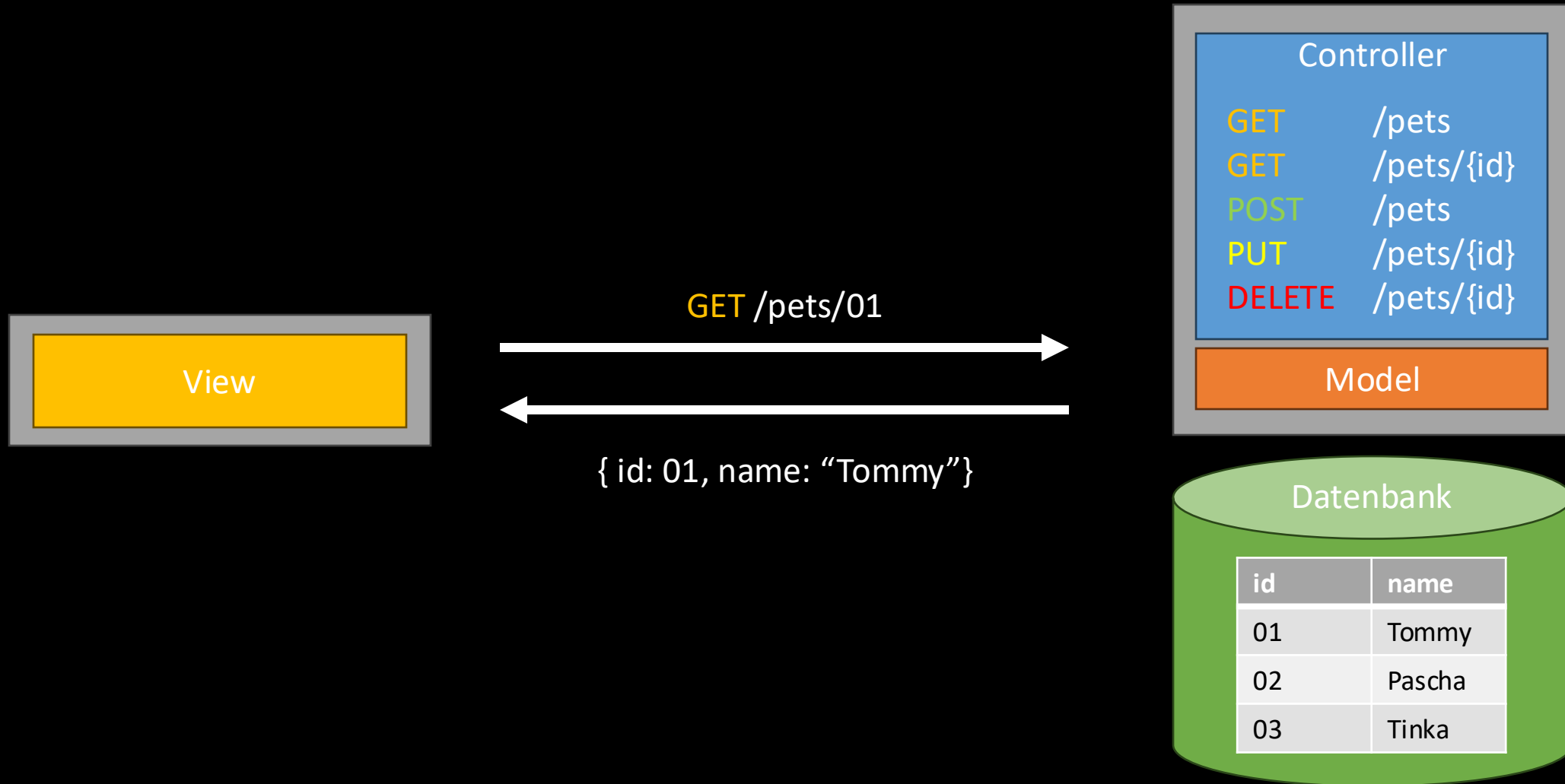
REST



REST



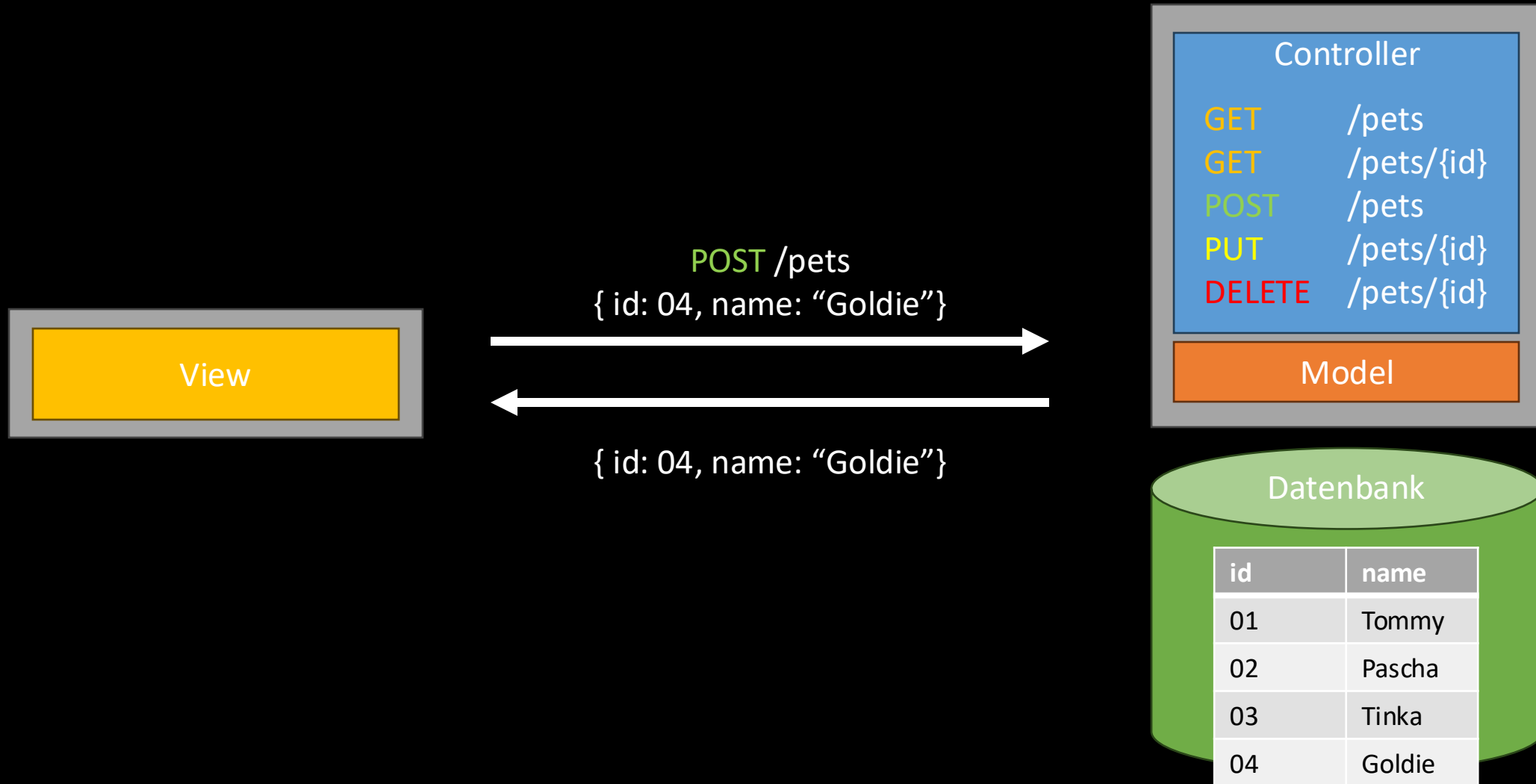
REST



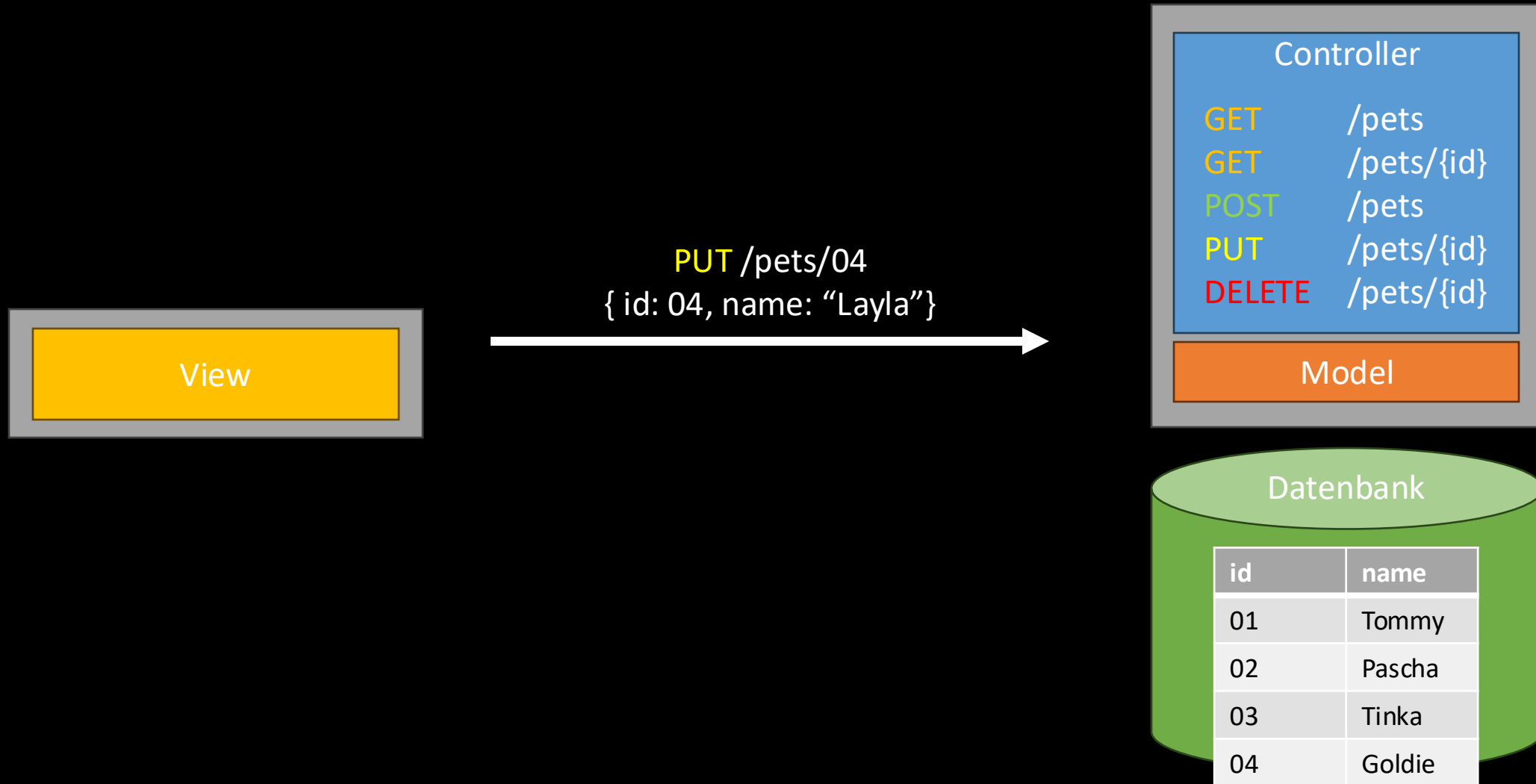
REST



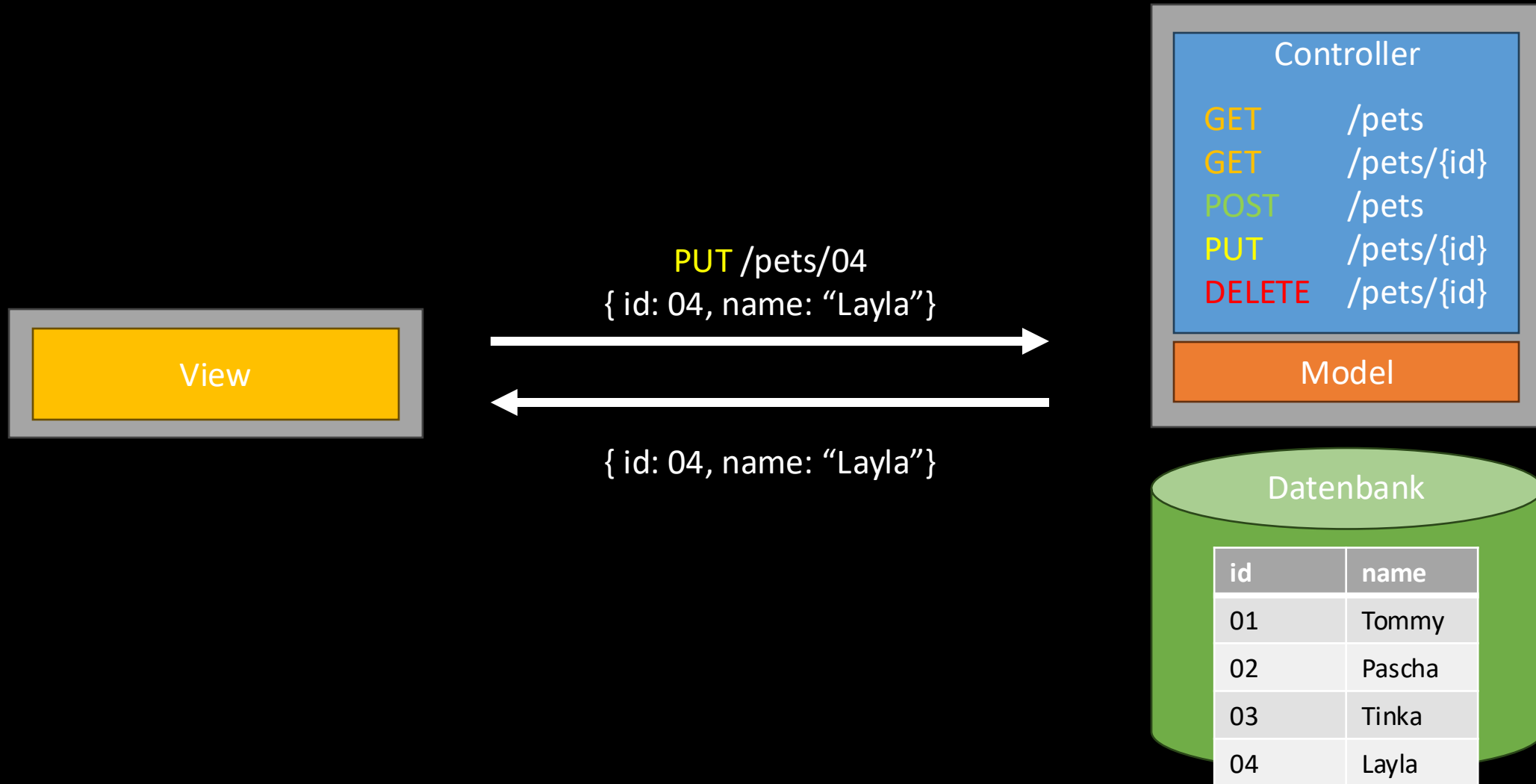
REST



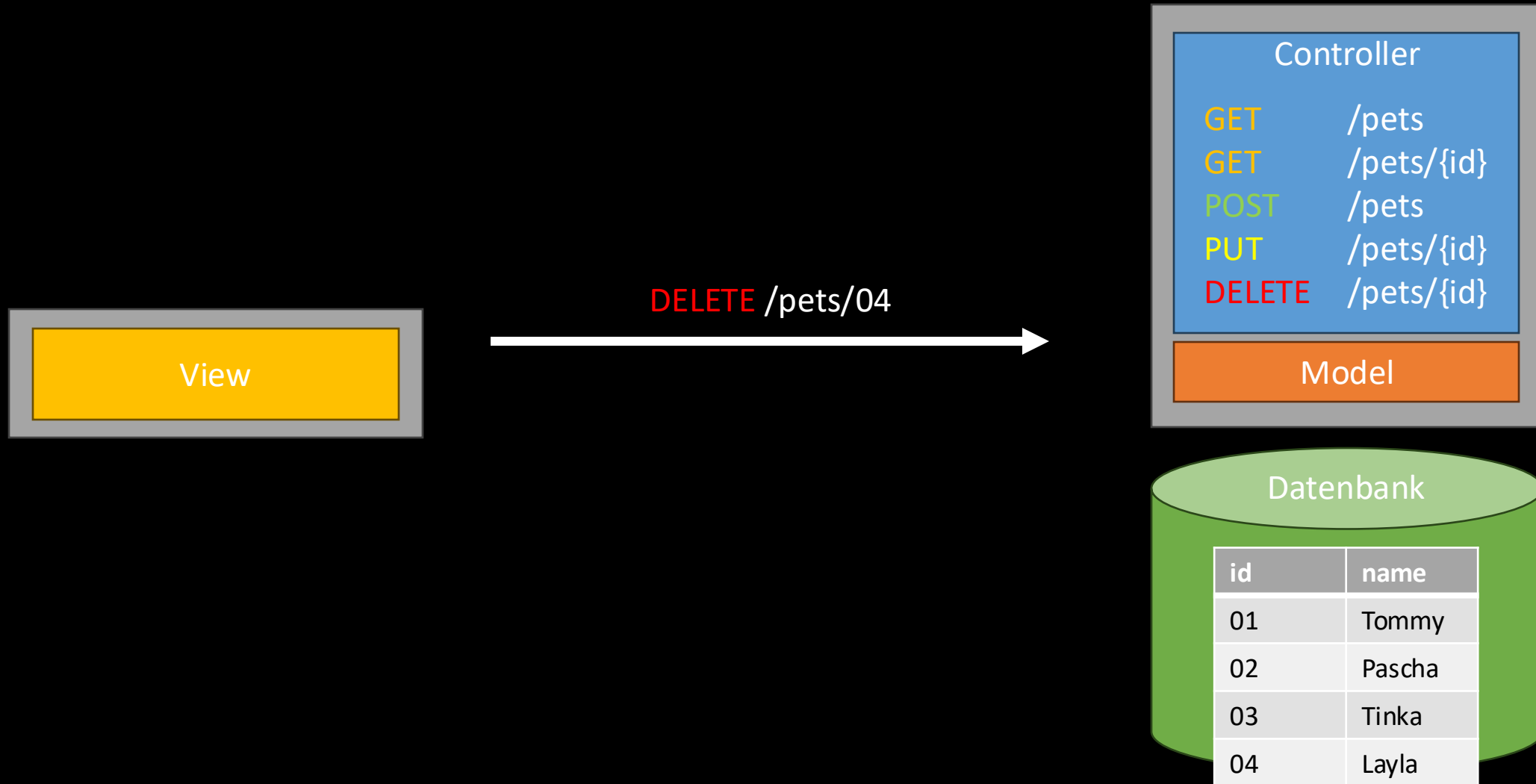
REST



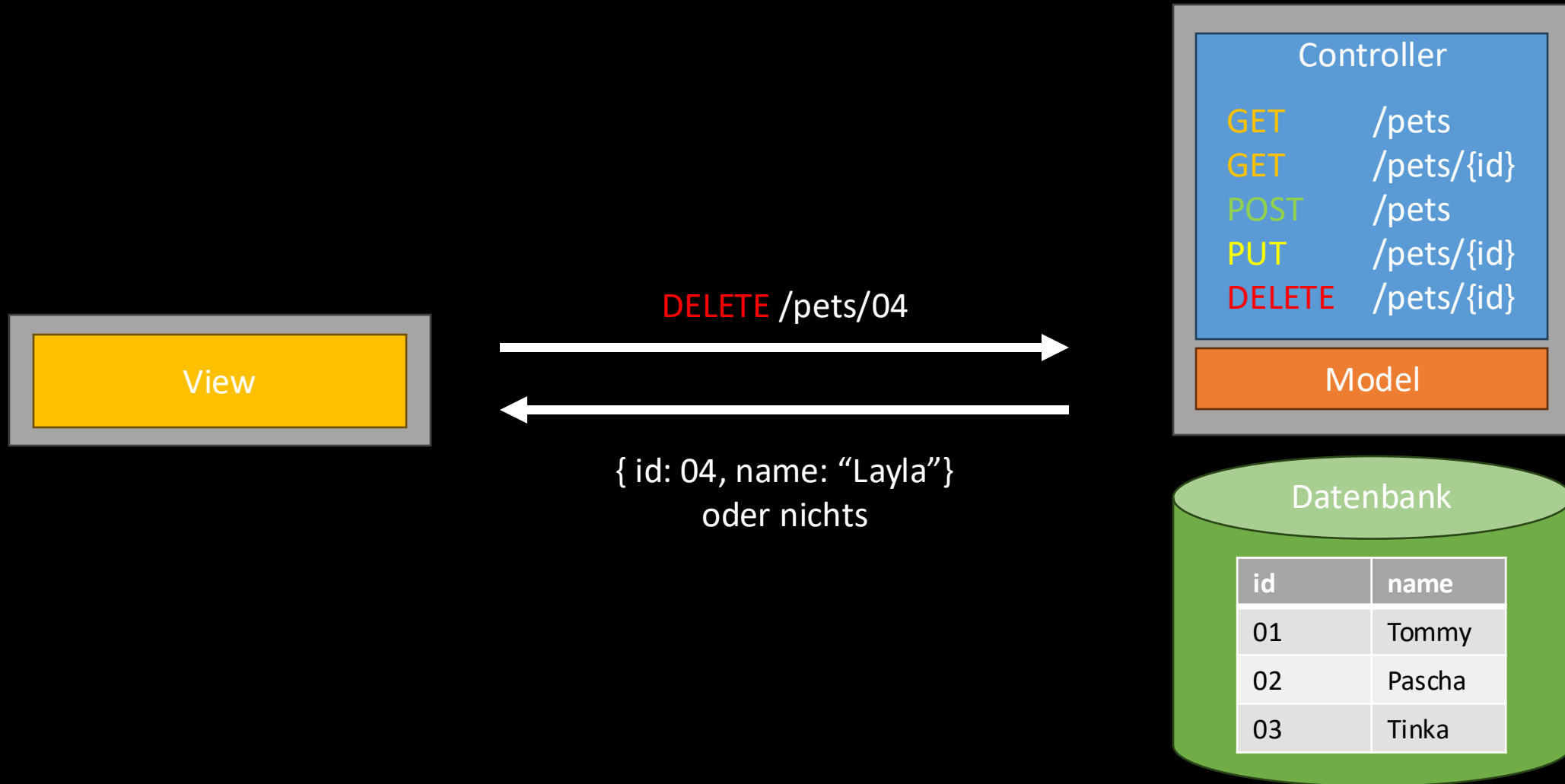
REST



REST



REST



REST

- Nachteile

... Was sind Nachteile?

REST

- Nachteile

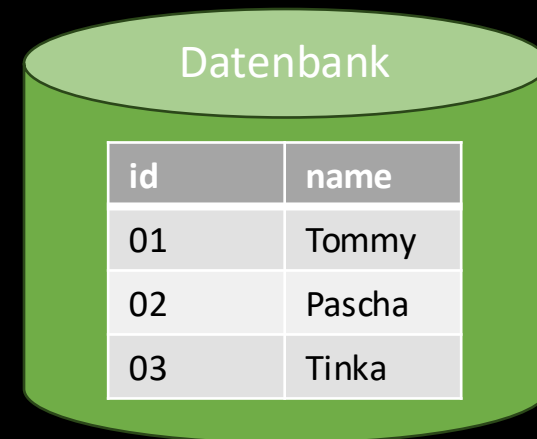
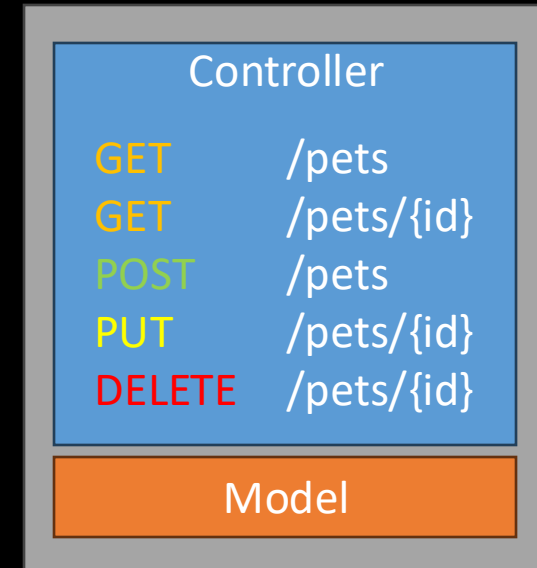
- Over-Fetching = zu viele Daten
- Under-Fetching = zu viele Anfragen nötig, um Ergebnis abzurufen
- Feste Struktur
- Aufwendiges API-Design (vor allem für komplexe Anfragen)

REST

Ich möchte nur die Namen von den Tieren bekommen, nicht die IDs



???



HTTP-Codes

- Status einer HTTP-Abfrage an eine API (o.Ä.)
- Methoden:
 - GET
 - POST
 - PUT
 - DELETE
 - ...
- 1xx: Information
- 2xx: Erfolg
- 3xx: Umleitung
- 4xx: Client-Fehler
- 5xx: Server-Fehler

Übung: HTTP-Bingo

- Folge den Anweisungen im Bingo-Repository
- Zeit: 5 Minuten

HTTP-Codes



400

Bad Request



500

Internal Server Error

...vermeiden, sondern spezifischere Codes nutzen!



100

Continue



103
Early Hints

1xx

Code	Nachricht	Bedeutung
100	Continue	Die laufende Anfrage an den Server wurde noch nicht zurückgewiesen. Der Client kann nun mit der potentiell sehr großen Anfrage fortfahren.
103	Early Hints	Wird zusammen mit dem „Link“-Header verwendet, um das Vorladen von Ressourcen zu ermöglichen, während der Server die finale Antwort noch vorbereitet.



200
OK



201
Created



202
Accepted



204
No Content

2xx

Code	Nachricht	Bedeutung
200	OK	Die Anfrage wurde erfolgreich bearbeitet und das Ergebnis der Anfrage wird in der Antwort übertragen.
201	Created	Die Anfrage wurde erfolgreich bearbeitet. Die angeforderte Ressource wurde vor dem Senden der Antwort erstellt. Das „Location“-Header-Feld enthält eventuell die Adresse der erstellten Ressource.
202	Accepted	Die Anfrage wurde akzeptiert, wird aber zu einem späteren Zeitpunkt ausgeführt. Das Gelingen der Anfrage kann nicht garantiert werden.
204	No Content	Die Anfrage wurde erfolgreich durchgeführt, die Antwort enthält jedoch bewusst keine Daten.



300
Multiple Choices



301

Moved Permanently



307

Temporary Redirect



308

Permanent Redirect

3xx

Code	Nachricht	Bedeutung
300	Multiple Choices	Die angeforderte Ressource steht in verschiedenen Arten zur Verfügung. Die Antwort enthält eine Liste der verfügbaren Arten. Das „Location“-Header-Feld enthält eventuell die Adresse der vom Server bevorzugten Repräsentation.
301	Moved Permanently	Die angeforderte Ressource steht ab sofort unter der im „Location“-Header-Feld angegebenen Adresse bereit (auch Redirect genannt). Die alte Adresse ist nicht länger gültig. Die Methode kann sich ändern.
307	Temporary Redirect	Die angeforderte Ressource steht vorübergehend unter der im „Location“-Header-Feld angegebenen Adresse bereit. Die alte Adresse bleibt gültig. Der Browser soll mit <i>derselben</i> Methode folgen wie beim ursprünglichen Request.
308	Permanent Redirect	Die angeforderte Ressource steht ab sofort unter der im „Location“-Header-Feld angegebenen Adresse bereit, die alte Adresse ist nicht länger gültig. Der Browser soll mit <i>derselben</i> Methode folgen wie beim ursprünglichen Request.



401
Unauthorized



402

Payment Required



403
Forbidden



404
Not Found



405

Method Not Allowed



406

Not Acceptable



408

Request Timeout

401-408

Code	Nachricht	Bedeutung
401	Unauthorized	Die Anfrage kann nicht ohne gültige Authentifizierung durchgeführt werden.
402	Payment Required	Bezahlung benötigt.
403	Forbidden	Die Anfrage wurde mangels Berechtigung des Clients nicht durchgeführt.
404	Not Found	Die angeforderte Ressource wurde nicht gefunden.
405	Method Not Allowed	Die Anfrage darf nur mit anderen HTTP-Methoden (zum Beispiel GET statt POST) gestellt werden. Gültige Methoden für die betreffende Ressource werden im „Allow“-Header-Feld der Antwort übermittelt.
406	Not Acceptable	Die angeforderte Ressource steht nicht in der gewünschten Form zur Verfügung. Gültige „Content-Type“-Werte können in der Antwort übermittelt werden.
408	Request Timeout	Innerhalb der vom Server erlaubten Zeitspanne wurde keine vollständige Anfrage des Clients empfangen.



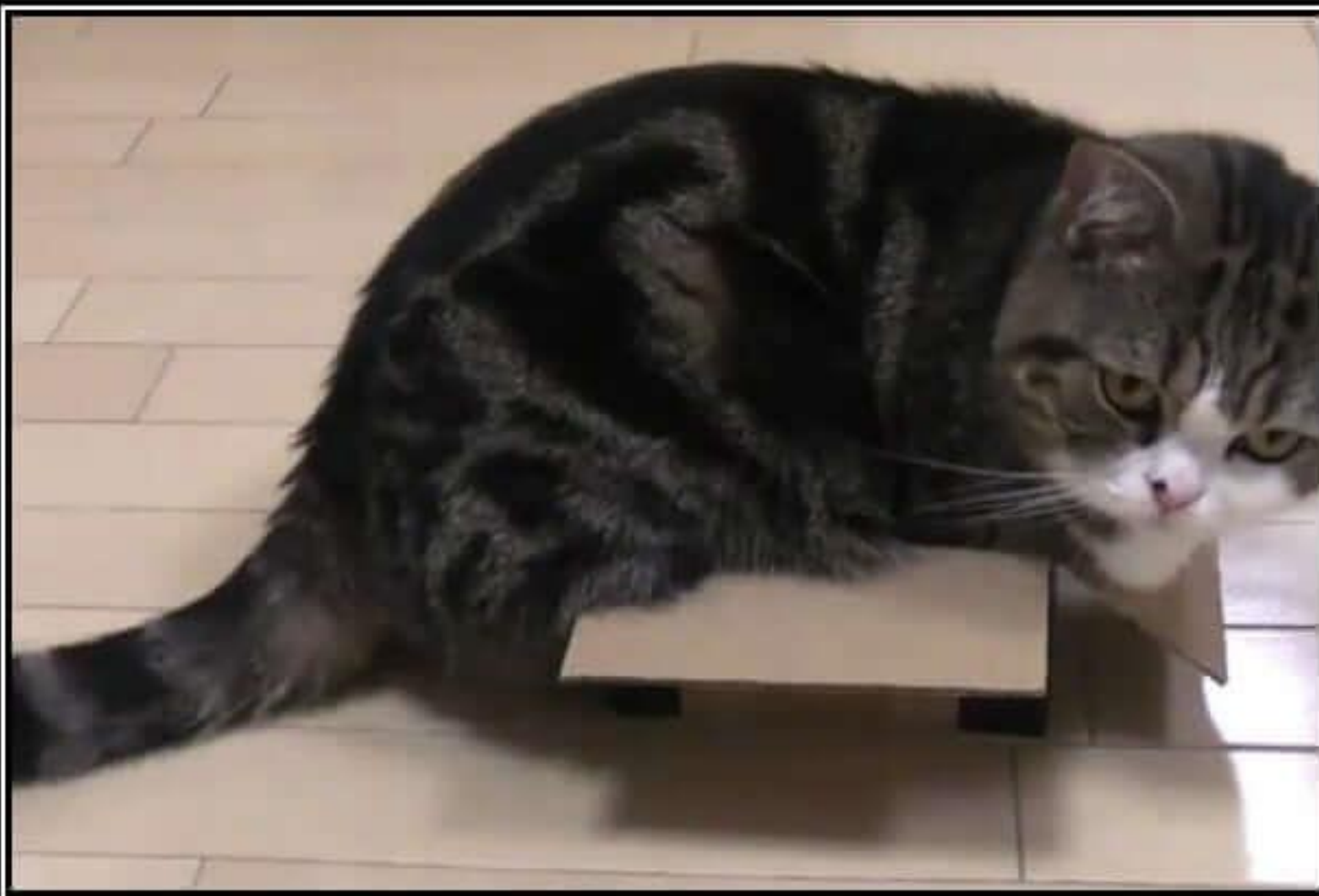
409

Conflict



410

Gone



413

Request Entity Too Large



415

Unsupported Media Type



418

I'm a teapot

409-418

Code	Nachricht	Bedeutung
409	Conflict	Die Anfrage wurde unter falschen Annahmen gestellt. Im Falle einer PUT-Anfrage kann dies zum Beispiel auf eine zwischenzeitliche Veränderung der Ressource durch Dritte zurückgehen.
410	Gone	Die angeforderte Ressource wird nicht länger bereitgestellt und wurde dauerhaft entfernt.
413	Payload Too Large	Die gestellte Anfrage war zu groß, um vom Server bearbeitet werden zu können.
415	Unsupported Media Type	Der Inhalt der Anfrage wurde mit ungültigem oder nicht erlaubtem Medientyp übermittelt.
418*	I'm A Teapot	Die angeforderte Ressource ist ein Teekessel.



420

Enhance Your Calm 



422

Unprocessable Entity



429

Too Many Requests



450

Blocked by Windows Parental Controls



451

Unavailable for Legal Reasons

420-451

Code	Nachricht	Bedeutung
420*	Enhance Your Calm*	Der Client hat zu viele Anfragen in einem bestimmten Zeitraum gesendet.
422	Unprocessable Entity	Die Anfrage kann wegen semantischer Fehler nicht bearbeitet werden.
429	Too Many Requests	Der Client hat zu viele Anfragen in einem bestimmten Zeitraum gesendet.
450*	Blocked By Windows Parental Controls	Dieser Statuscode soll darauf hinweisen, dass die angeforderte Ressource aufgrund der aufgesetzten Kindersicherung nicht verfügbar ist.
451	Unavailable For Legal Reasons	Dieser Statuscode soll darauf hinweisen, dass die angeforderte Ressource aufgrund von gesetzlichen Bestimmungen nicht verfügbar ist.



501

Not Implemented



502

Bad Gateway



503

Service Unavailable



504

Gateway Timeout



507

Insufficient Storage



530
Site Frozen

5xx

Code	Nachricht	Bedeutung
501	Not Implemented	Die Funktionalität, um die Anfrage zu bearbeiten, wird von diesem Server nicht bereitgestellt.
502	Bad Gateway	Der Server konnte seine Funktion als Gateway oder Proxy nicht erfüllen, weil er seinerseits eine ungültige Antwort erhalten hat.
503	Service Unavailable	Der Server steht temporär nicht zur Verfügung, zum Beispiel wegen Überlastung oder Wartungsarbeiten.
504	Gateway Timeout	Der Server konnte seine Funktion als Gateway oder Proxy nicht erfüllen, weil er innerhalb einer festgelegten Zeitspanne keine Antwort von seinerseits benutzten Servern oder Diensten erhalten hat.
507	Insufficient Storage	Die Anfrage konnte nicht bearbeitet werden, weil der Speicherplatz des Servers dazu derzeit nicht mehr ausreicht.
530*	Site Frozen	Die Anfrage konnte nicht bearbeitet werden, weil der Service "eingefrohren" ist.

GraphQL

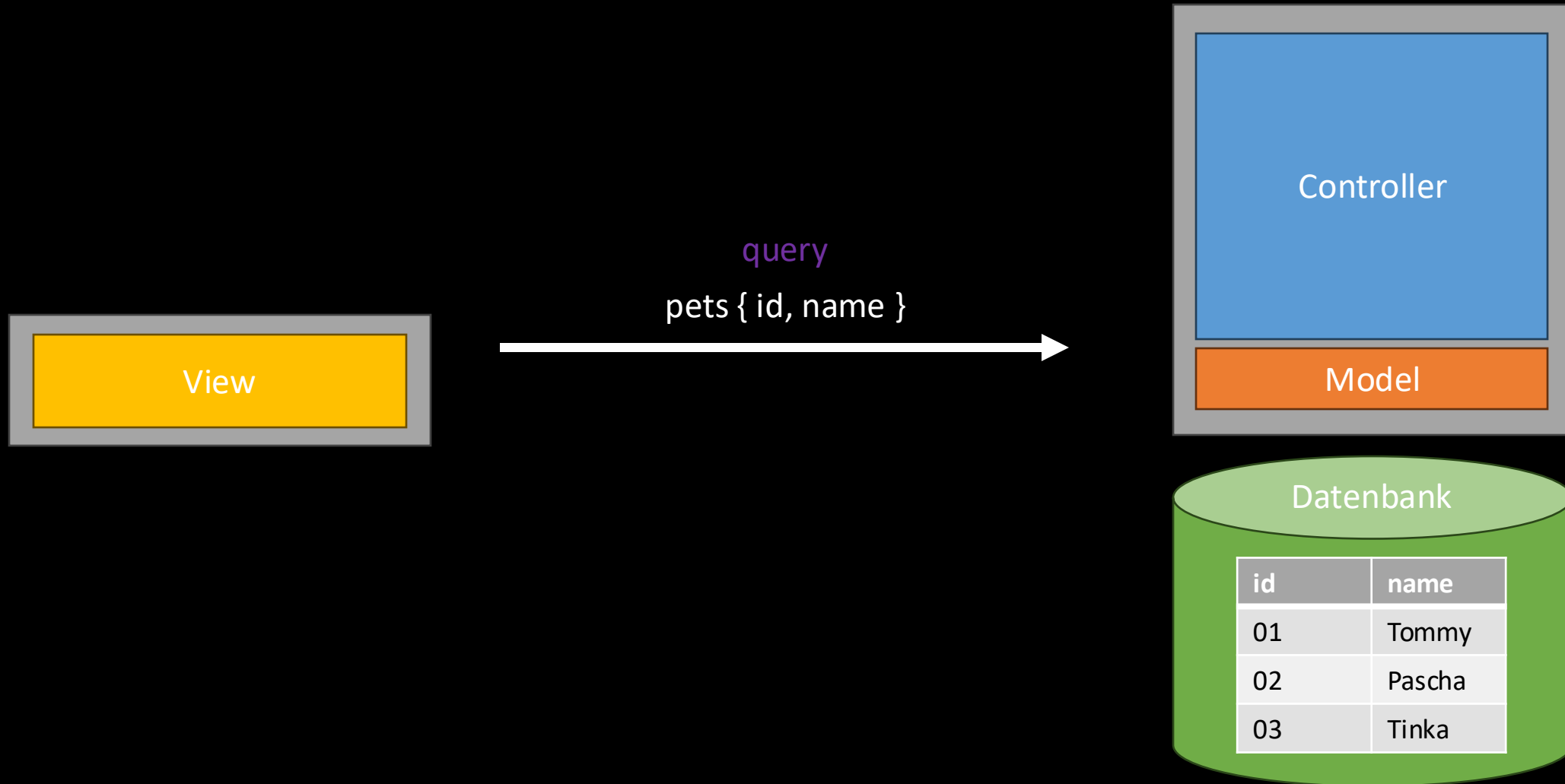
Was ist GraphQL?

Wie unterscheidet es sich zu REST?

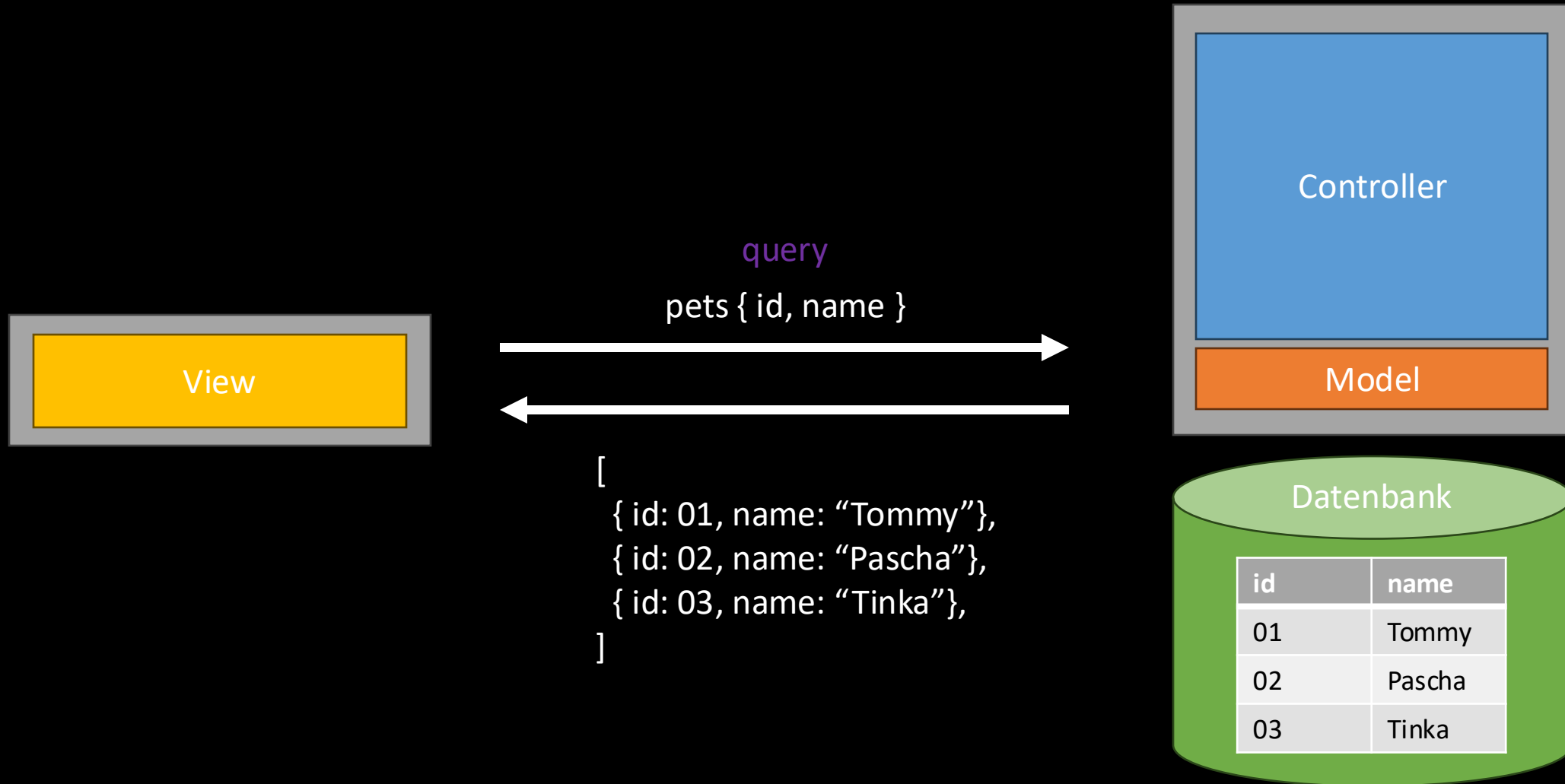
GraphQL

- 2005 von Facebook entwickelt
- Abfragesprache für APIs
- Anfrage von spezifischen Daten, kein* Over- oder Unter-Fetching
- Ein einziger Endpunkt
- „Queries“ & „Mutations“

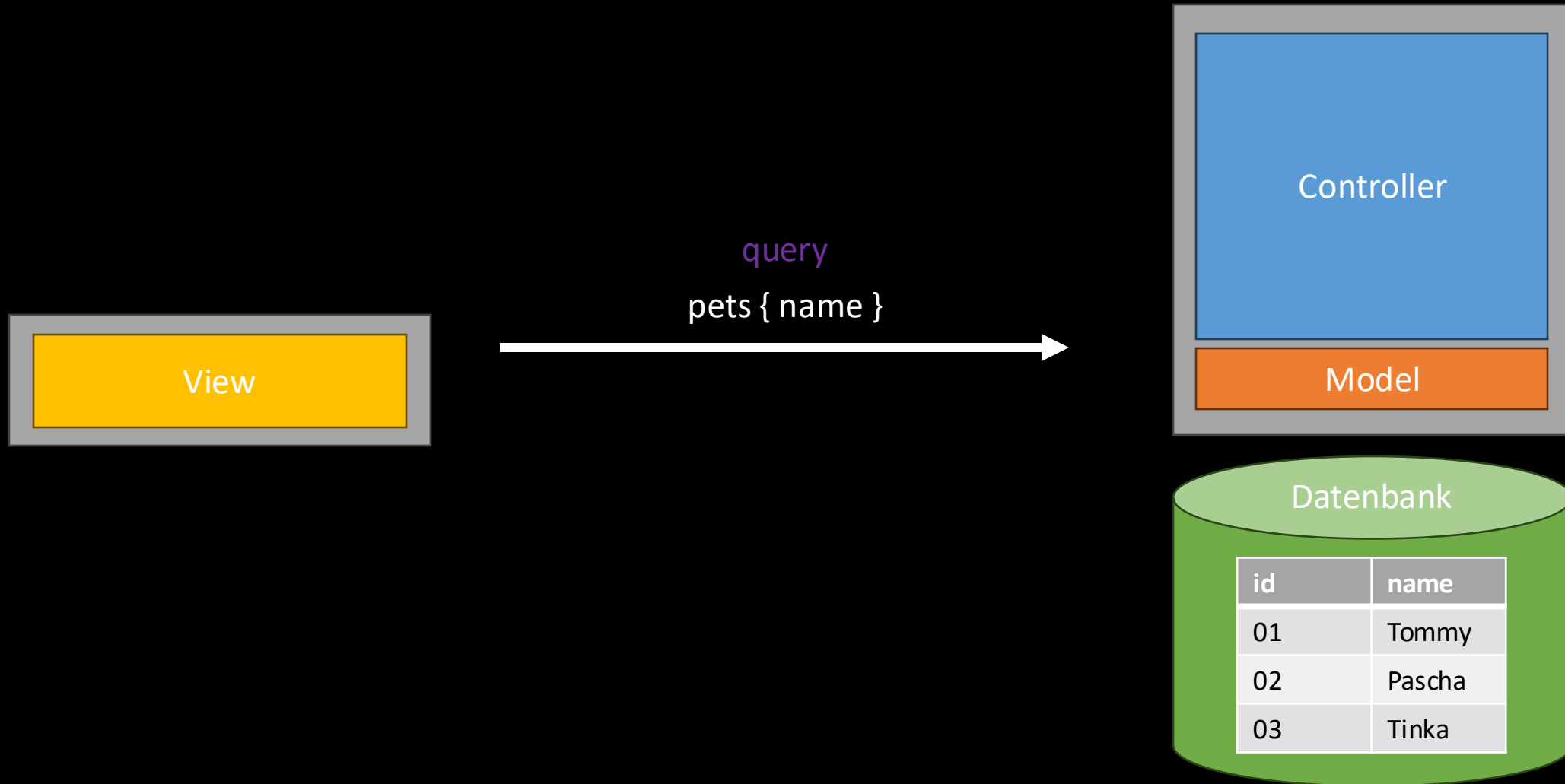
GraphQL



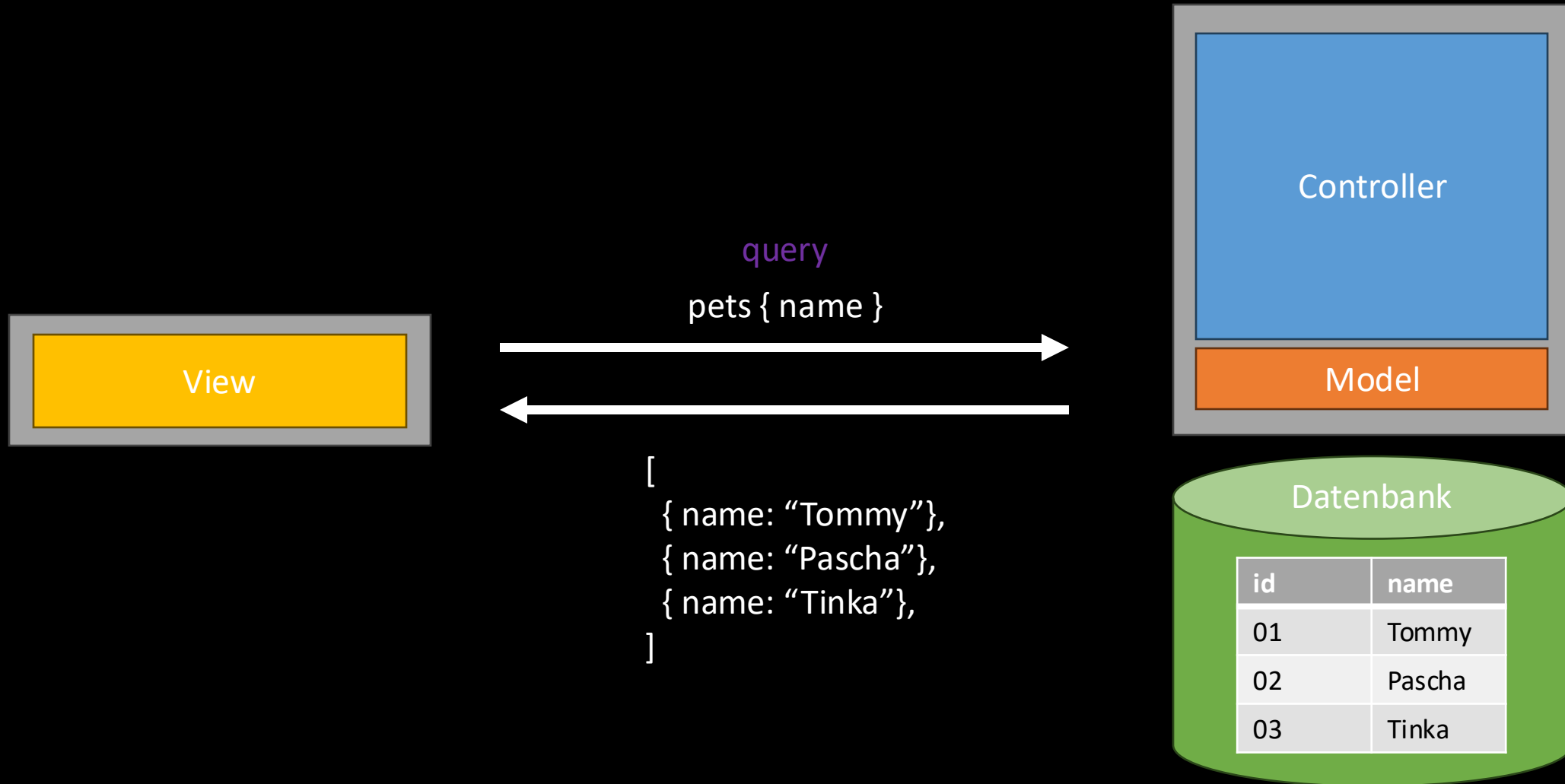
GraphQL



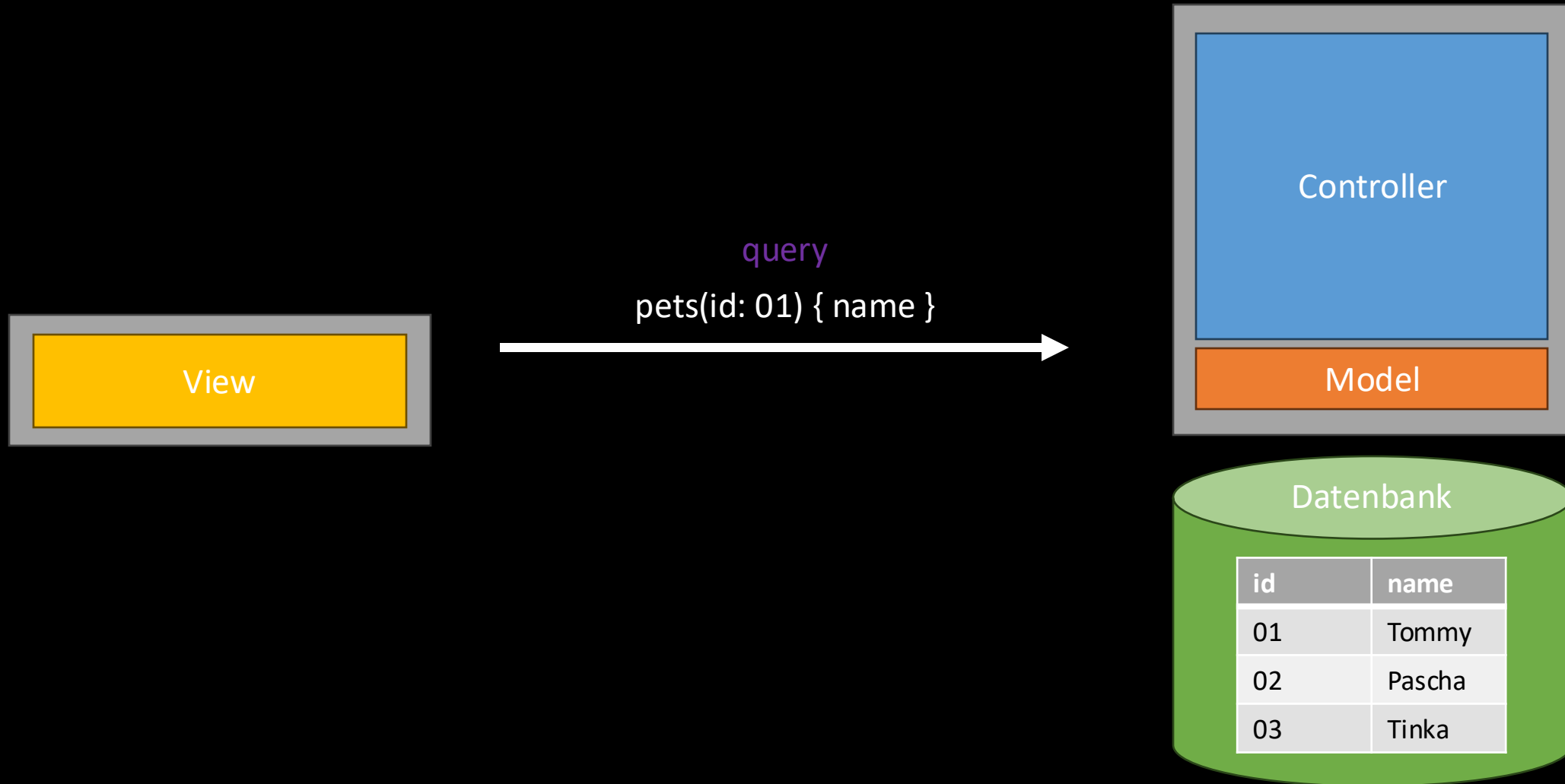
GraphQL



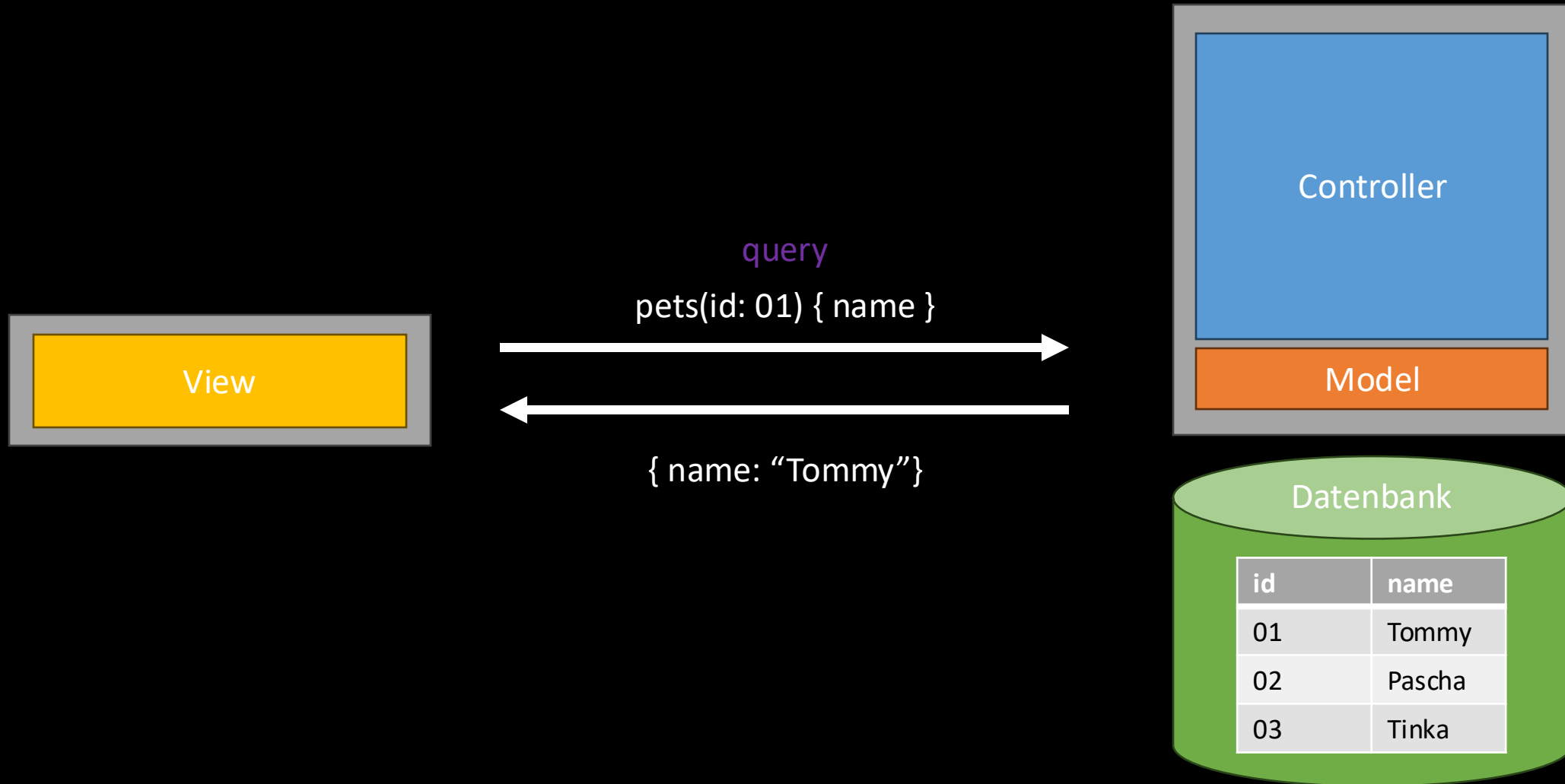
GraphQL



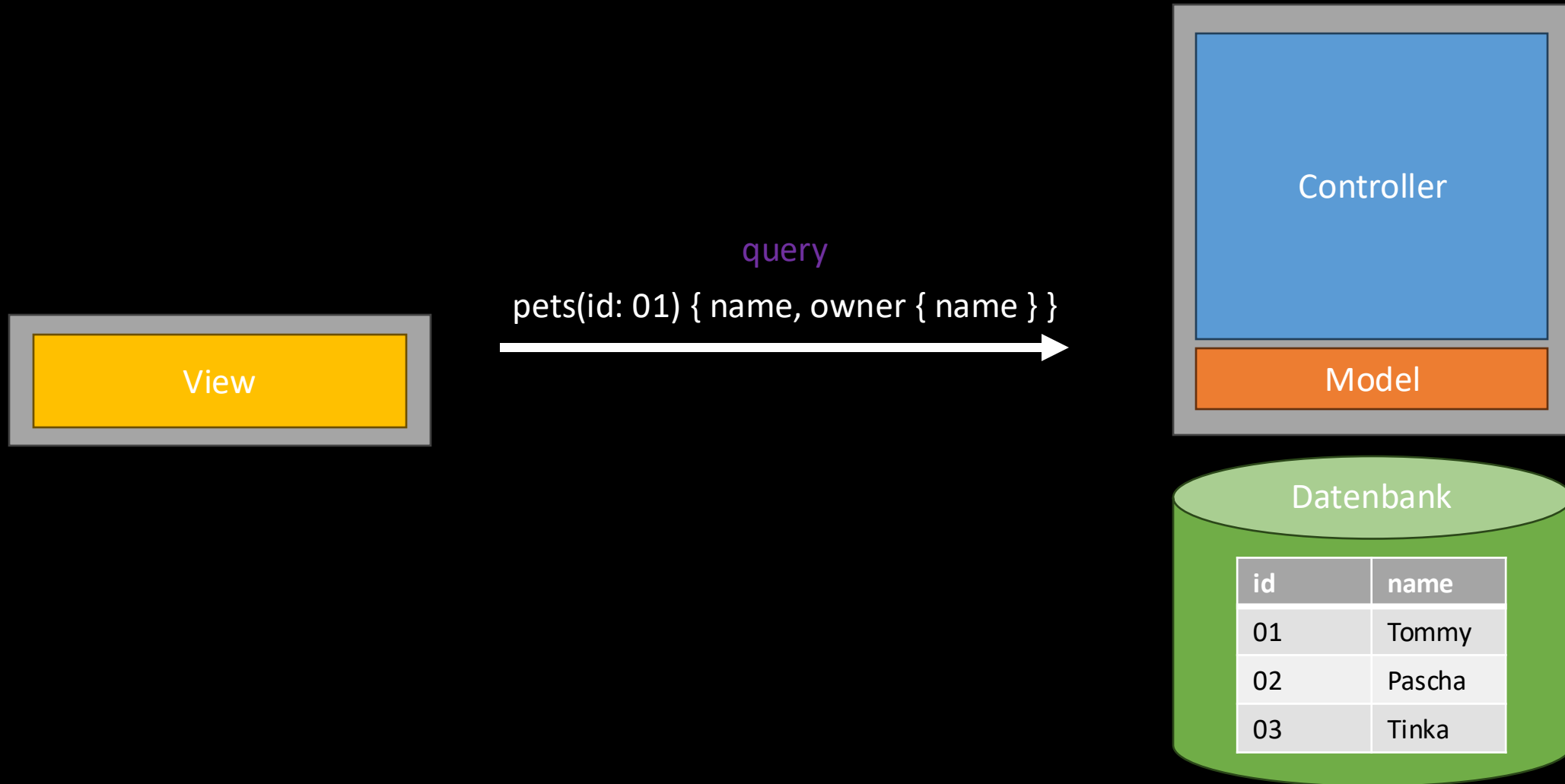
GraphQL



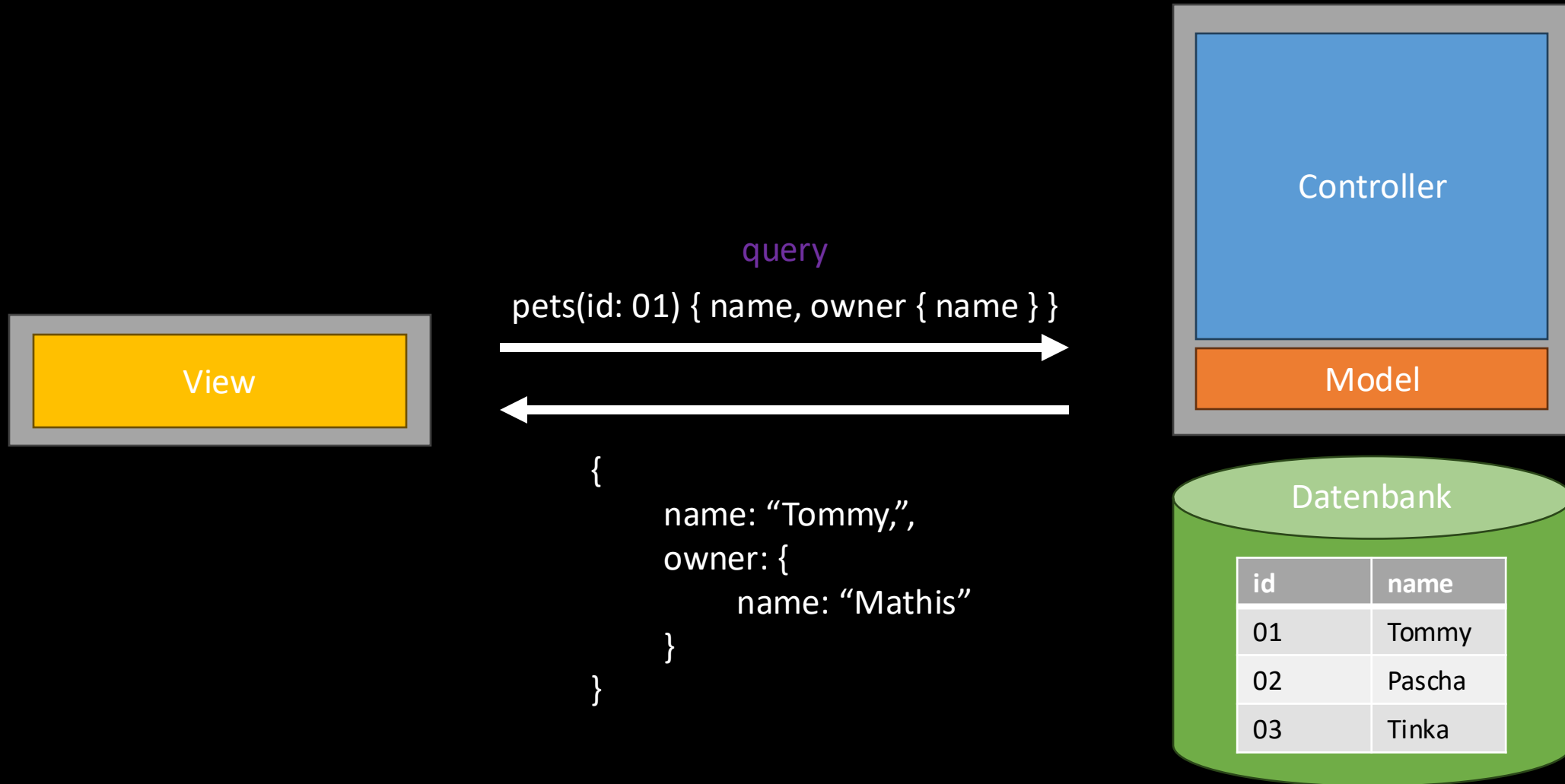
GraphQL



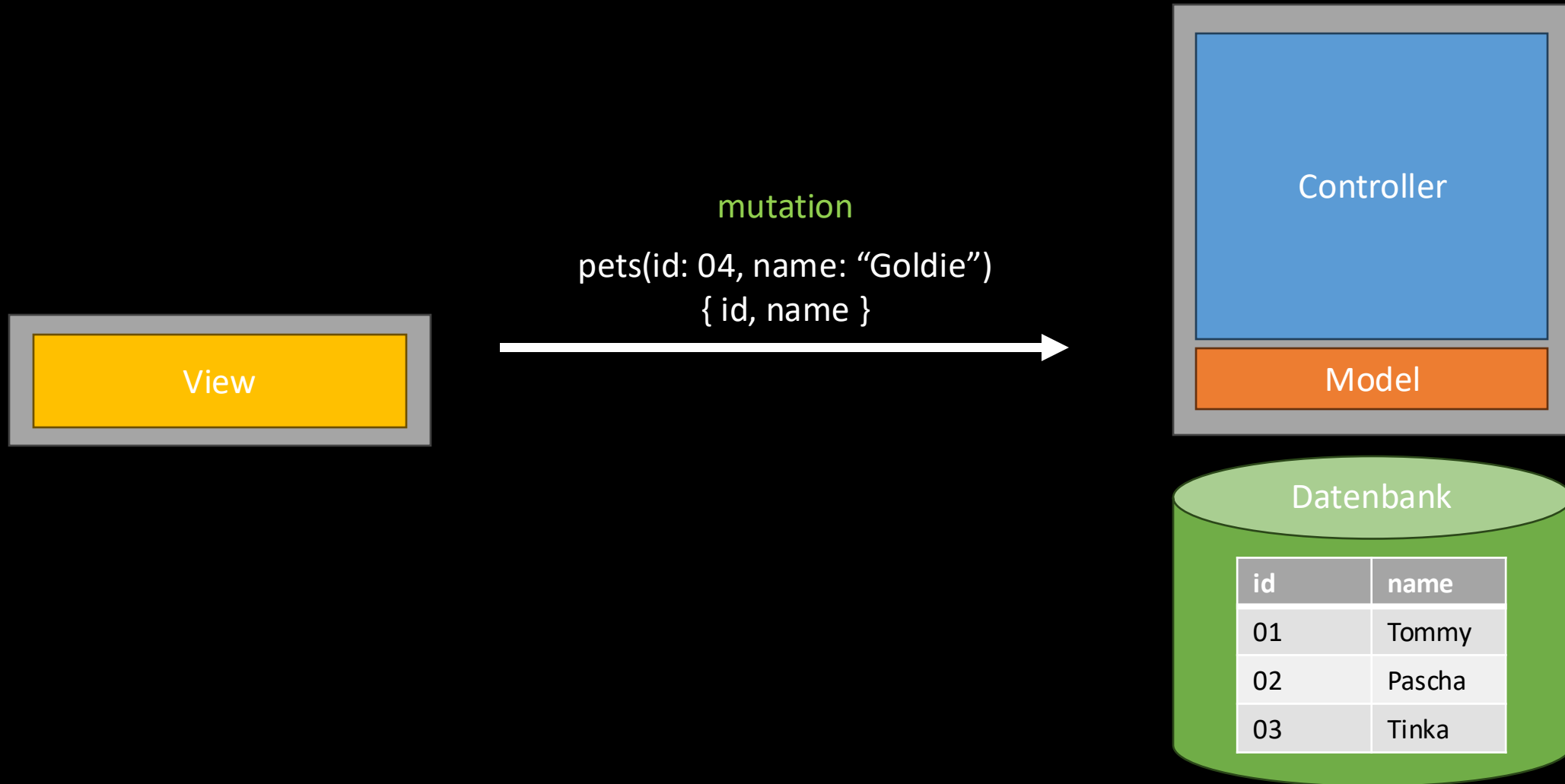
GraphQL



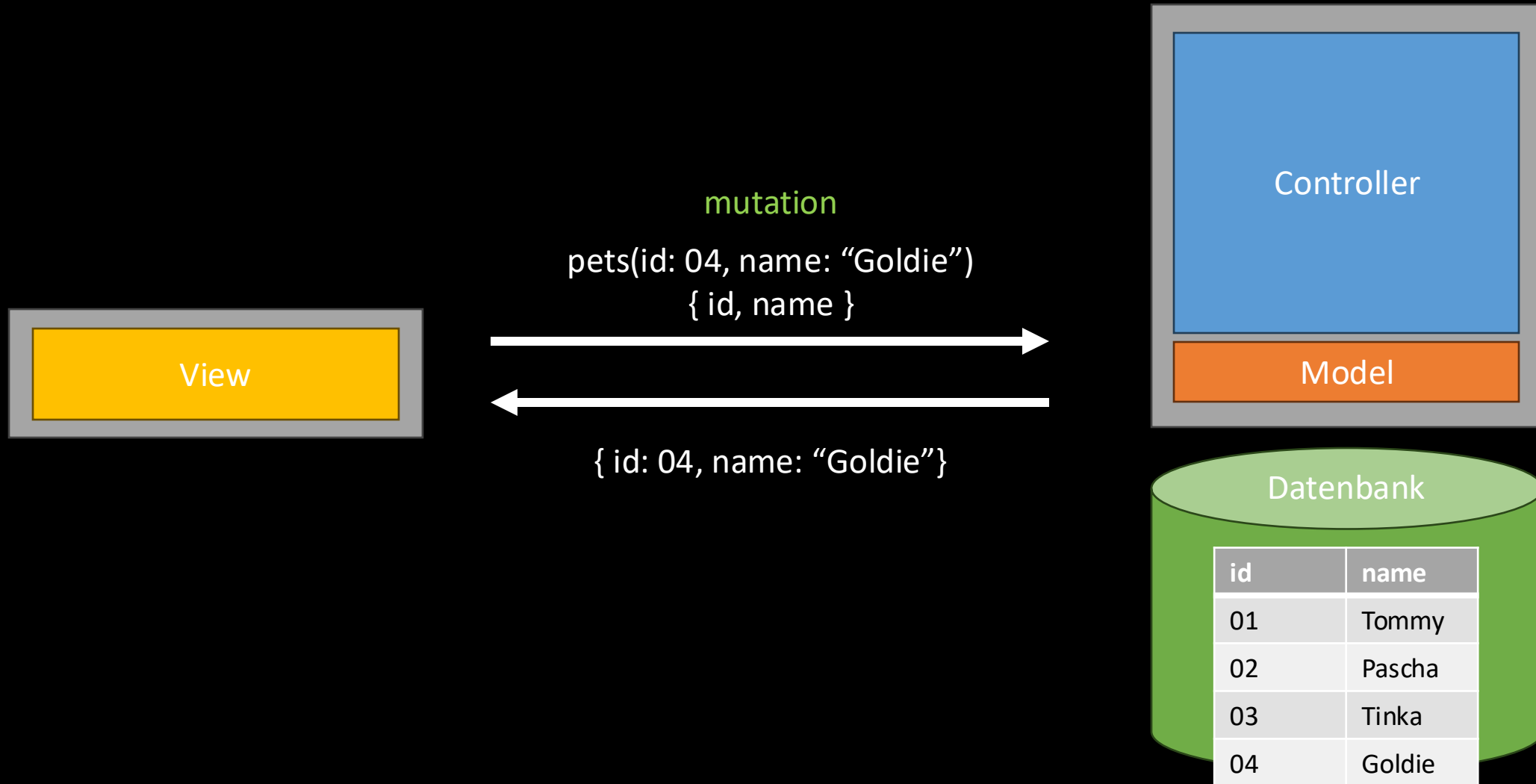
GraphQL



GraphQL



GraphQL



REST vs GraphQL

REST

- Mehrere Endpunkte
- Starre Struktur
- Einfaches Caching
- Im Hintergrund potenziell einfach

GraphQL

- Ein Endpunkt
- Flexible Anfragen
- Kein Caching
- Im Hintergrund recht komplex

OData

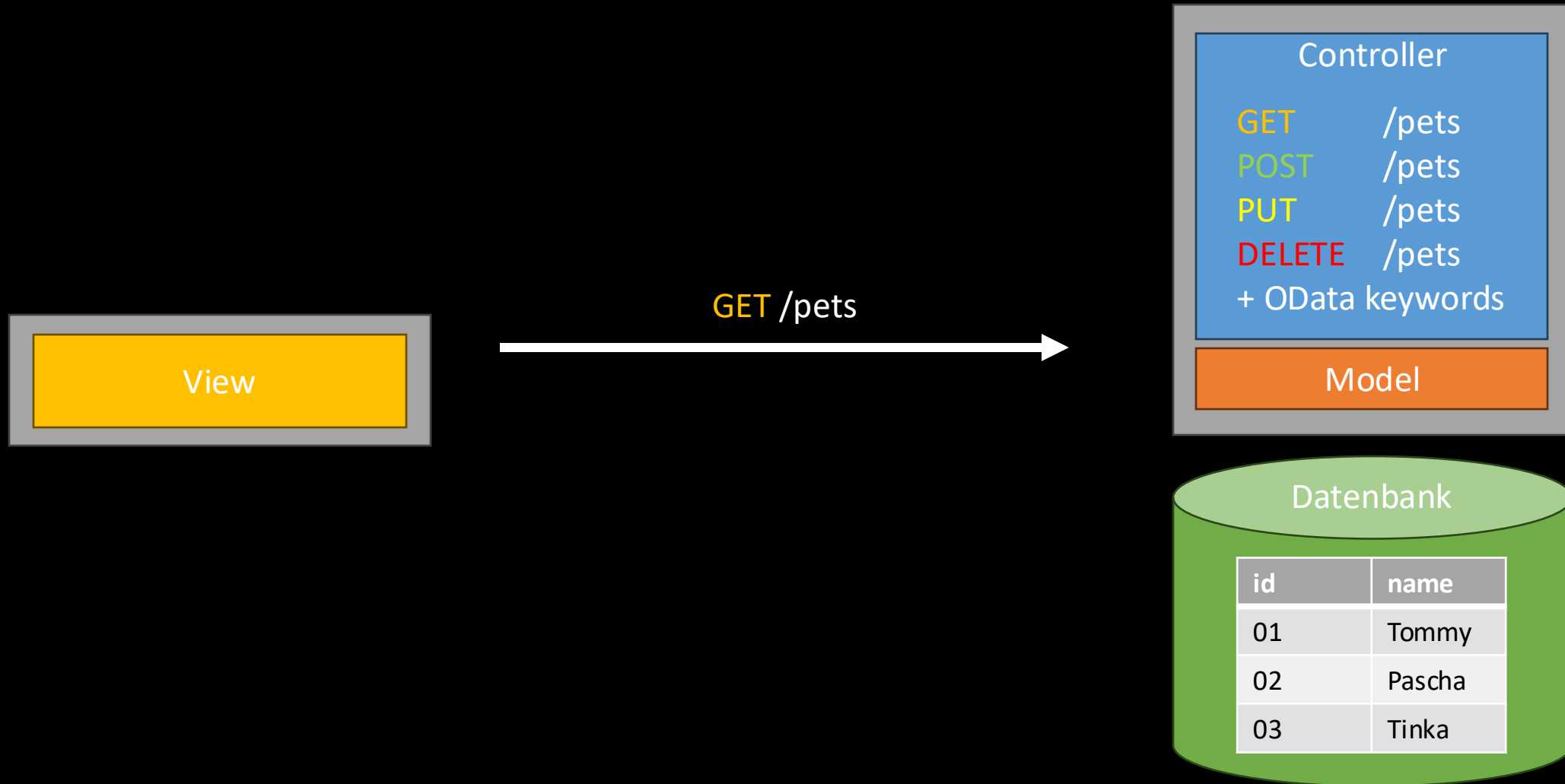
Was ist OData?

Wie unterscheidet es sich zu REST?

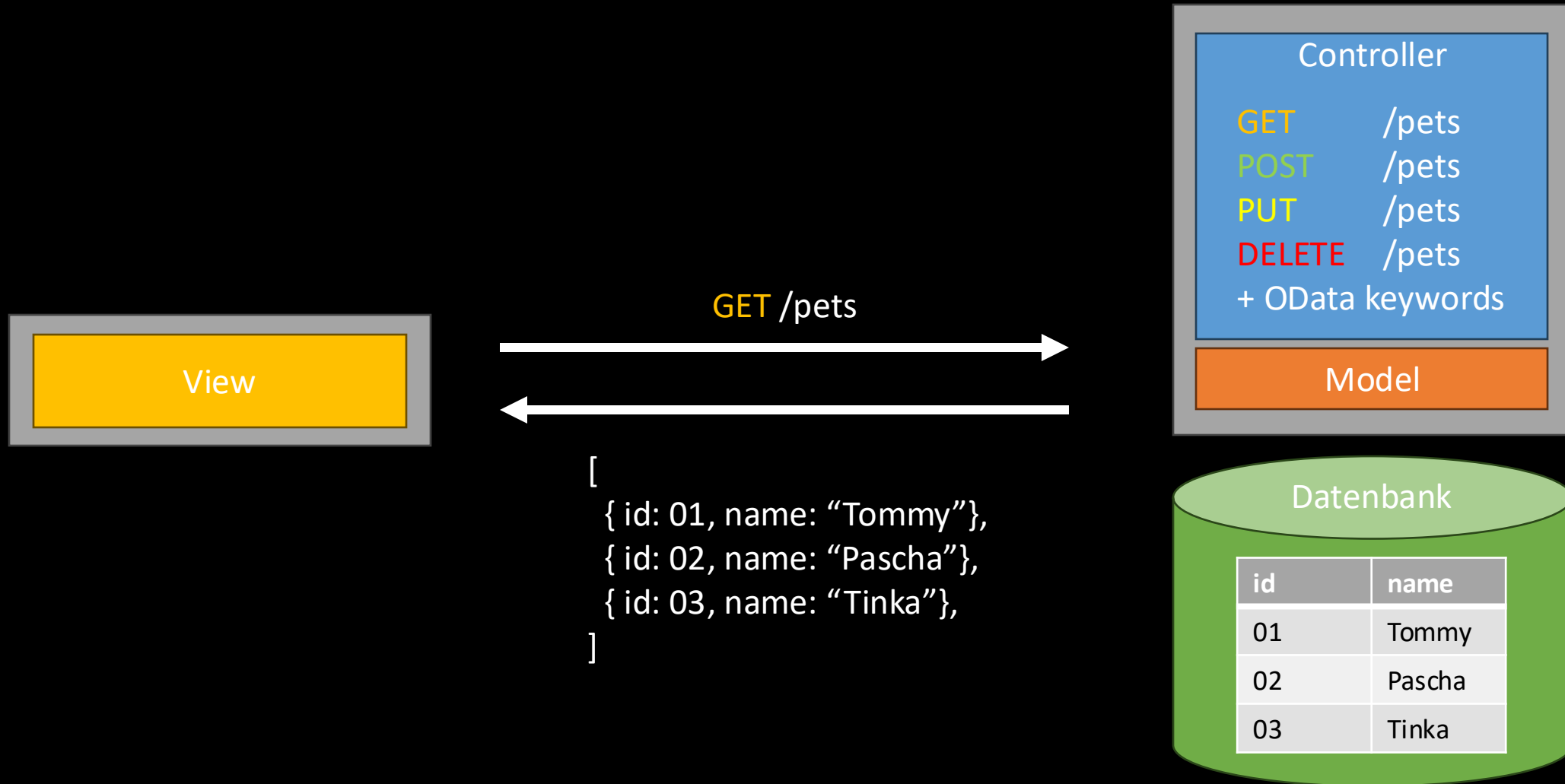
OData

- Entwickelt von Microsoft
- Viel im SAP-Umfeld genutzt
- Abfragesprache für APIs
- „Erweiterung“ von REST mit Metadaten und Query-Operatoren
- Verschiedene Versionen:
 - ehemals v2
 - aktuell v4

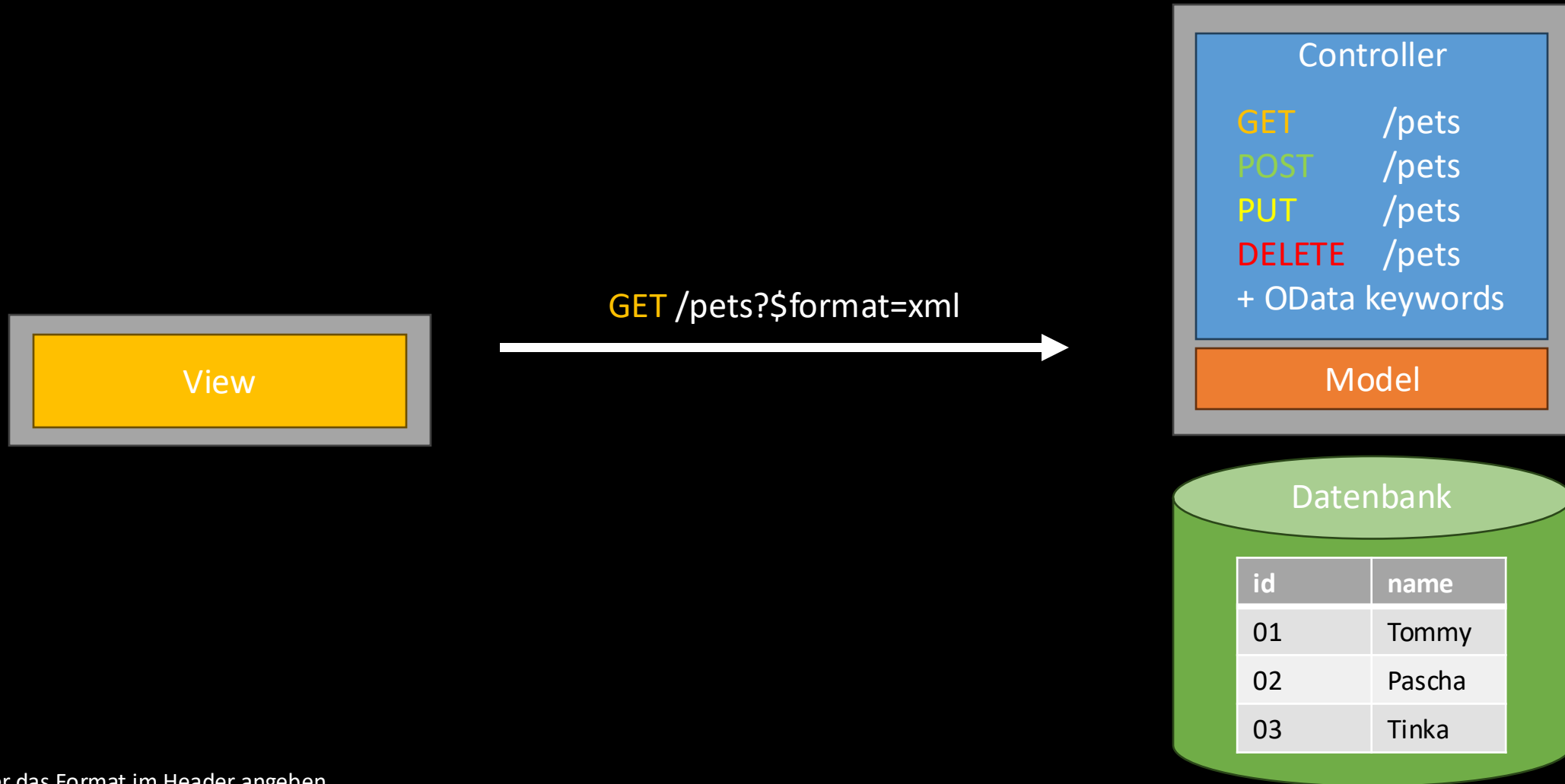
OData



OData



OData



OData

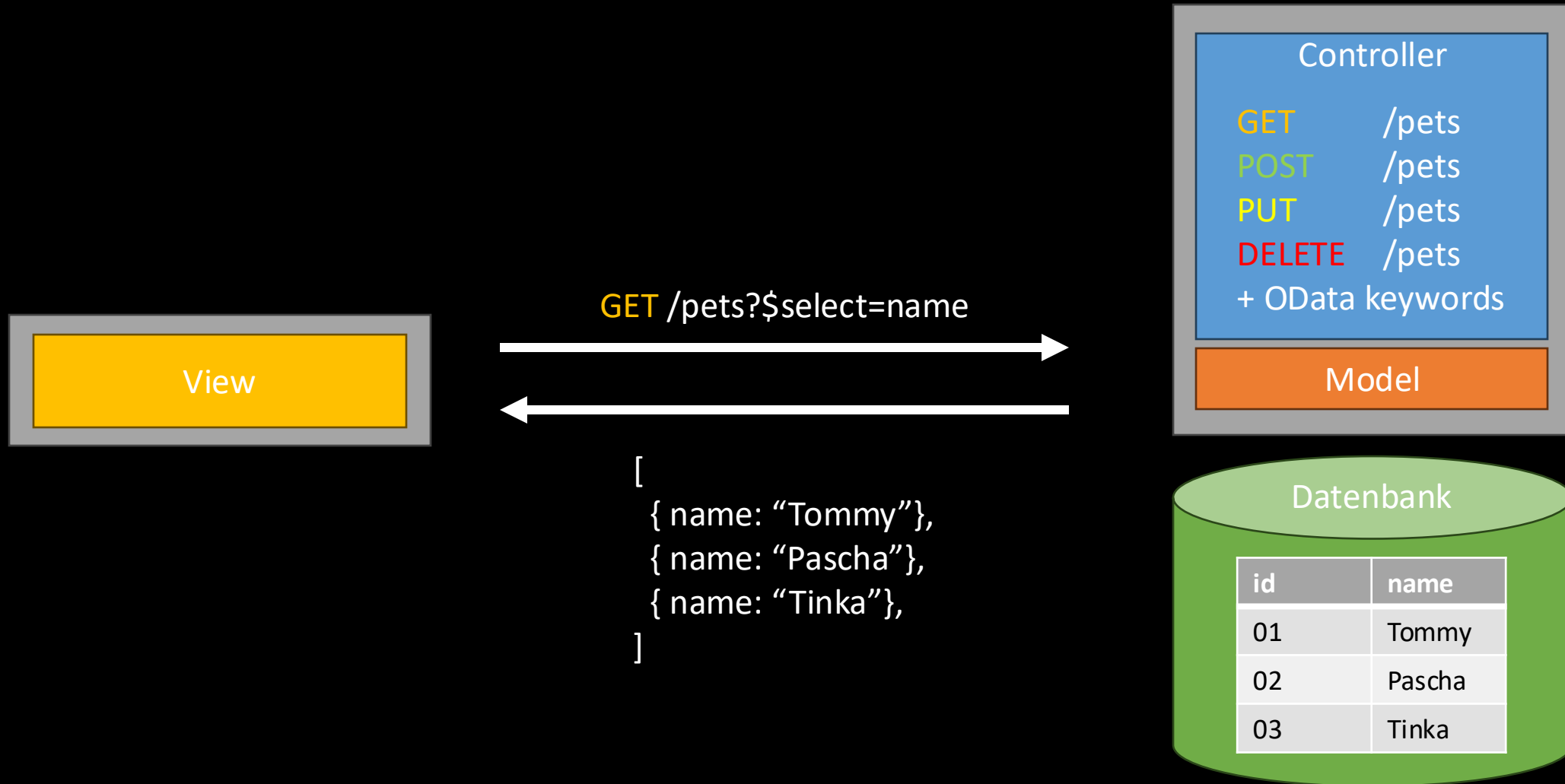


*oder das Format im Header angeben

OData



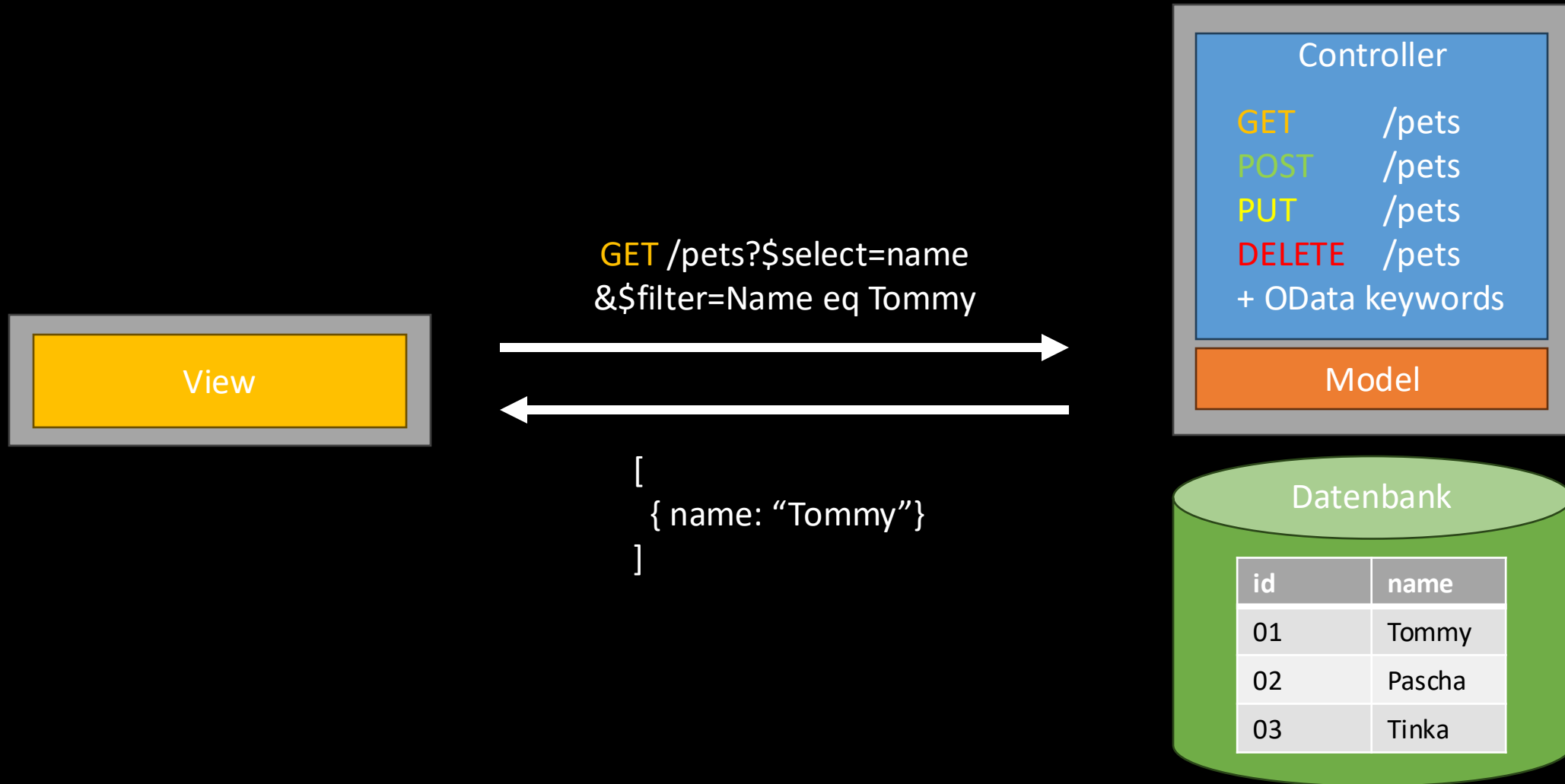
OData



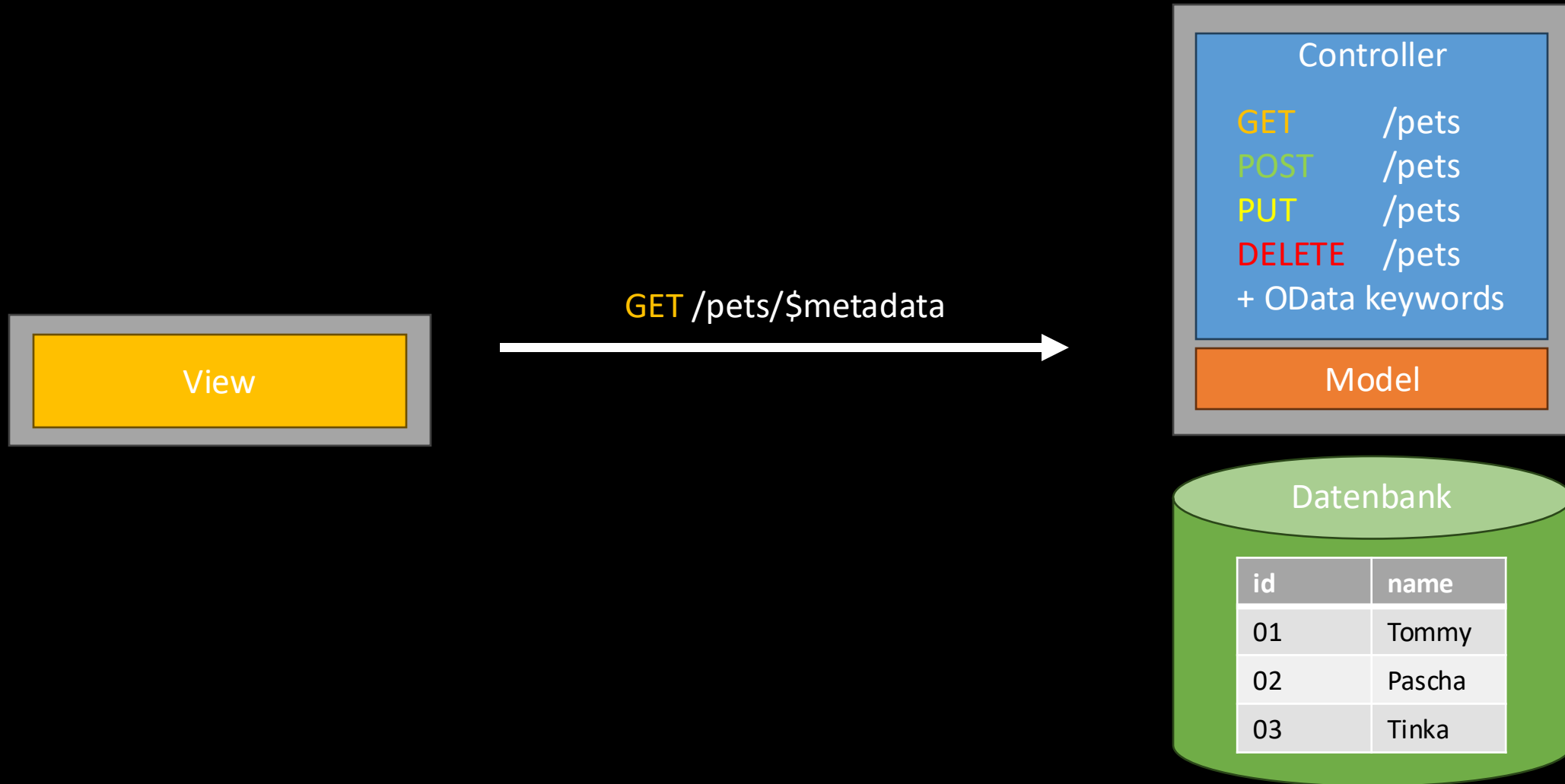
OData



OData



OData



OData



REST vs OData

REST

- Keine strenge Spezifikation
- Keine eingebauten Filter und Queries
- JSON als Format

OData

- Standardisierte Regeln und Metadaten
- Einheitliche Query-Parameter, z.B. \$select oder \$filter
- Wahl zwischen XML und JSON

Übung: REST vs GraphQL

- Teste die REST-API im Ordner „rest“ (npm run start)
 - 1x GET
 - 1x POST
 - 1x PUT
 - 1x DELETE
- Teste die GraphQL-API der rnv
 - Bau einen Abfahrtsmonitor für deine Haltestelle
- Zeit: 30 min

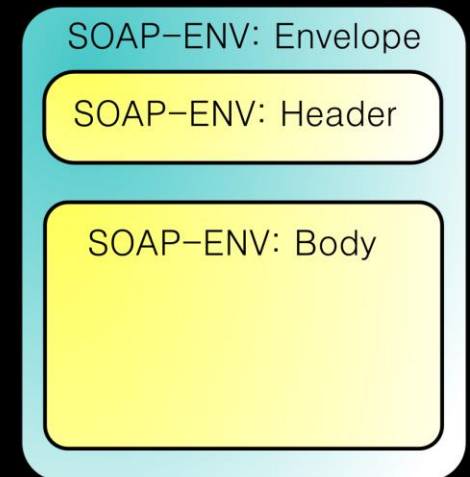
SOAP

Was ist SOAP?

Wie unterscheidet es sich zu REST?

SOAP

- SOAP = Simple Object Access Protocol (früher)
- 1999 von Microsoft entwickelt
- „Die Abkürzung SOAP wird offiziell ab Version 1.2 nicht mehr als Akronym gebraucht, da es erstens (subjektiv) keineswegs einfach (Simple) ist [...]“
- Besteht aus einem Envelope mit Header und Body
- Nutzt XML als Darstellung
- Basiert auf WSDL (Web Services Description Language) und XSD (XML Schema Definition)
- Arbeit mit WS-*, einer Sammlung von Regeln, wie z.B. Sicherheit



SOAP

POST /pets

```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:GetPetById>
      <pet:id>01</pet:id>
    </pet:GetPetById>
  </soap:Body>
</soap:Envelope>
```

View

Controller

/PetService mit
SOAP Actions:
- GetPetById
- UpdatePet

Model

Datenbank

id	name
01	Tommy
02	Pascha
03	Tinka

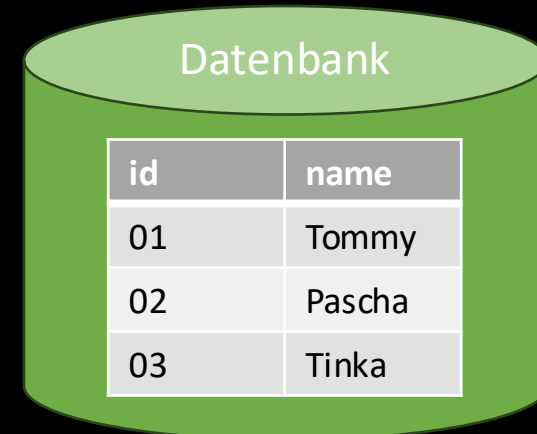
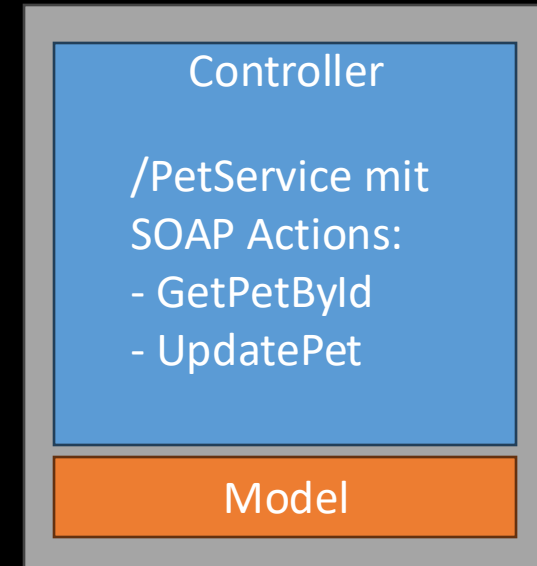
SOAP

POST /pets

```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:GetPetById>
      <pet:id>01</pet:id>
    </pet:GetPetById>
  </soap:Body>
</soap:Envelope>
```



```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:GetPetByIdResponse>
      <pet:id>01</pet:id>
      <pet:name>Tommy</pet:name>
    </pet:GetPetByIdResponse >
  </soap:Body>
</soap:Envelope>
```



SOAP

POST /pets

```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:UpdatePet>
      <pet:id>01</pet:id>
      <pet:name>Goldie</pet:name>
    </pet:UpdatePet >
  </soap:Body>
</soap:Envelope>
```

View

Controller

/PetService mit
SOAP Actions:
- GetPetById
- UpdatePet

Model

Datenbank

id	name
01	Tommy
02	Pascha
03	Tinka

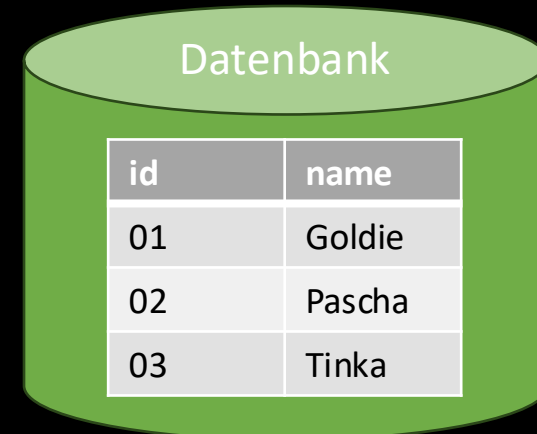
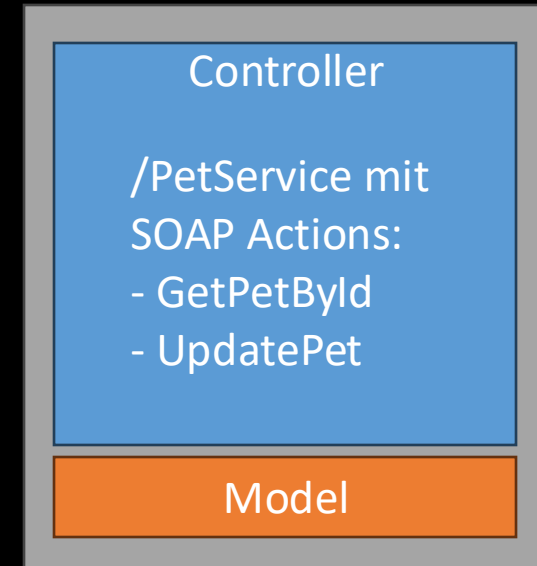
SOAP

POST /pets

```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:UpdatePet>
      <pet:id>01</pet:id>
      <pet:name>Goldie</pet:name>
    </pet:UpdatePet >
  </soap:Body>
</soap:Envelope>
```



```
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
  xmlns:pet="http://example.com/petservice">
  <soap:Header/>
  <soap:Body>
    <pet:UpdatePetResponse>
      <pet:id>01</pet:id>
      <pet:name>Goldie</pet:name>
    </pet:UpdatePetResponse >
  </soap:Body>
</soap:Envelope>
```



REST vs SOAP

REST

- Leichtgewichtig
- Kein striktes Schema
- GET, POST, PUT, DELETE, ...
- Schnell implementiert
- Wenige Formalitäten

SOAP

- Viel Overhead(!)
- WSDL beschreib genaue Schnittstellen und Datentypen
- Fast immer POST
- Recht komplex zu implementieren
- Viele Standards

Websockets

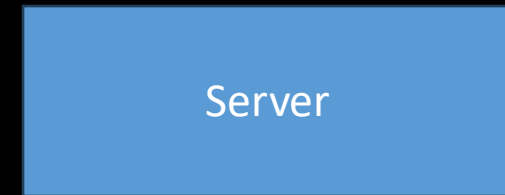
Wie funktionieren Websockets?

Websockets

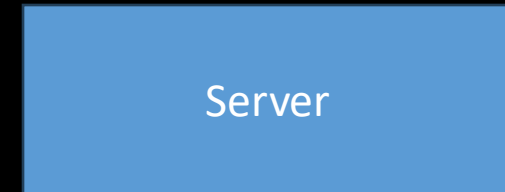
- Protokoll für bidirektionale Kommunikation über eine , TCP-Verbindung
- Entwickelt für Echtzeit-Anwendungen (z.B. Chat-Apps)
- Startet mit einem Handshake und wechselt dann zum WebSocket-Protokoll
- Verbindung besteht durchgehend

Websockets vs HTTP

Websockets



HTTP (z.B. REST)



RFC

Wie funktioniert RFC?

RFC

- Aufruf einer Funktion in einem entfernten SAP-System
- Verbreitet in SAP-Systemen
- 3 Arten von RFC
 - sRFC: Synchron
 - tRFC: Asynchron (“transactional”)
 - qRFC: Mit einer Warteschlange (“queued”)
- Hinterlegte Logik (z.B. in Form von Funktionsbausteinen in ABAP), die Parameter entgegennimmt und Ergebnisse liefert
- SAP Gateway Vermittelt zwischen aufrufendem Client und Zielsystem (RFC-Destination)

RFC

- Nutzung: Datenaustausch
- Wie nutzt man RFC?
 - Funktionsbaustein in ABAP entwickeln
 - Als “remote enabled” kennzeichnen
 - Von anderen SAP-Systemen aufrufen
 - Oder: durch externe Tools (z.B. SAP JCo (Java Connector)) aufrufen

RFC

```
CALL FUNCTION 'Z_UPDATE_PET'  
  DESTINATION 'RFC_TARGET_SYSTEM'  
  EXPORTING  
    iv_pet_id = '01'  
    iv_pet_name = 'Goldie'  
  IMPORTING  
    ev_message = lv_message  
  EXCEPTIONS  
    system_failure = 1  
    communication_failure = 2  
    others = 3.  
  
IF sy-subrc = 0.  
  WRITE: / 'Pet wurde erfolgreich aktualisiert:', lv_message.  
ELSE.  
  WRITE: / 'Fehler beim RFC-Aufruf, SY-SUBRC:', sy-subrc.  
ENDIF.
```

RFC

```
CALL FUNCTION 'Z_UPDATE_PET'
  DESTINATION 'RFC_TARGET_SYSTEM'
  EXPORTING
    iv_pet_id = '01'
    iv_pet_name = 'Goldie'
  IMPORTING
    ev_message = lv_message
  EXCEPTIONS
    system_failure = 1
    communication_failure = 2
    others = 3.

IF sy-subrc = 0.
  WRITE: / 'Pet wurde erfolgreich aktualisiert:', lv_message.
ELSE.
  WRITE: / 'Fehler beim RFC-Aufruf, SY-SUBRC:', sy-subrc.
ENDIF.
```



```
FUNCTION Z_UPDATE_PET.

  DATA: lv_subrc TYPE sy-subrc.

  UPDATE ztable_pet
  SET name = iv_pet_name
  WHERE id = iv_pet_id.

  lv_subrc = sy-subrc.

  IF lv_subrc = 0.
    EV_MESSAGE = |Pet { IV_PET_ID } mit Name "{ IV_PET_NAME }" erfolgreich
    aktualisiert.|.
  ELSE.
    EV_MESSAGE = |Fehler beim Aktualisieren von Pet { IV_PET_ID }. (SUBRC={ lv_subrc })|.
  ENDIF.

ENDFUNCTION.
```

REST vs SOAP

Protokoll	Datenformat	Anwendungsfälle	Vorteile	Nachteile
REST	JSON (+ andere)	Web-/Mobile-APIs	Einfach & weit verbreitet	Under- & Overfetching
GraphQL	JSON	Flexibler Datenabruf	Vermeidet Under- & Overfetching	Komplex
OData	JSON & XML	Datenzentrierte REST-APIs (z.B. SAP)	Reichhaltige Abfragesyntax	Overhead und nicht so flexibel
SOAP	XML	Enterprise-Integration & Legacy	Sicherheit & WS-* Standards	Aufwendig & viel Overhead
Websockets	Text & Binär	Echtzeit-Anwendungen	Echtzeit	Verbindung muss bestehen
RFC	SAP-spezifisch	SAP-Integration	Direkte Funktionsaufrufe	Meist auf SAP-Kontexte beschränkt

Danke!

Mathis Neunzig
mathis.neunzig@gmail.com

